Building Knowledge Graphs: From Sources to Verifiable Queries

Mick Cooney

10 November 2026

Table of Contents

1 Introduction **2**

2 Design the Graph Around Questions **4**
2.1 What an Edge Lets You Ask 4
2.2 Define the Northbridge Evidence Boundary 5
2.3 Turn the Boundary Into Answer Contracts 7

3 Build an Evidence Ledger **9**
3.1 Preserve Mentions and Resolve Entities 9
3.2 Give Claims and Assertions Separate Roles 12
3.3 Transform Source Wording Into Reviewable Rows 14
3.4 Follow the Evidence Back to Its Source 17

4 Interpret Time, Conflict, and Uncertainty **18**
4.1 Distinguish Supported Periods From Unknown Duration 18
4.2 Keep Domain Time and Recorded Time Apart 19
4.3 Recognise Disagreement at the Right Grain 20

5 Answer Questions From the Graph **21**
5.1 Look Up a Version and an Owner 22
5.2 Inspect Assertions and Their Evidence 23
5.3 Compare the Same Proposition Across Sources 24
5.4 Follow a Bounded Path 25

6 Diagnose and Correct the Answer **27**
6.1 Trace a Failure Through Stages 27
6.2 Run the Three Controlled Passes 30

7 Govern Disclosure and Approve a Release **33**
7.1 Separate Authorised Review From Ordinary Use 33
7.2 Version Releases as Evidence Products 36
7.3 Evaluate the Permitted Outputs 38

7.4 Check the Release Regression 41

8 Transfer the Method to Literature and Dependencies 44
8.1 Literature Evidence for a Research Question 44
8.1.1 Define the Sources and Model 44
8.1.2 Work the Query 48
8.1.3 Correct the Selected Source Version 49
8.1.4 Audit the Evidence and Interpretation 51
8.2 Installed Dependencies and Advisory Applicability 52
8.2.1 Define the Inventory and Model 53
8.2.2 Traverse and Test Advisory Applicability 56
8.2.3 Correct the Inventory and Replay 57
8.2.4 Audit Scope, Boundaries, and Missing Evidence 58
8.3 Compare the Modelling Decisions 61

9 Analyse a Positive Projection 62
9.1 Construct the View 63
9.2 Work the Path and Component Calculations 64
9.3 Return From Topology to Evidence 65

10 Practise the Reasoning 66
10.1 Questions 66
10.2 Worked Answers 67

11 Storage and Implementation Reference 69
11.1 RDF, Evidence Resources, and Constraints 69
11.2 Property Graphs and Serving Choices 71
11.3 Companion Implementation Notes 72

12 Further Reading 73

1 Introduction

On 15 March 2024, two Northbridge records give different accounts of the travel policy for project Atlas. The policy register has version 1 in force for a period that includes that day. The Atlas minutes explicitly deny that version 1 applies on that day and affirm that version 2 does. If you draw an arrow from each version to Atlas, the diagram can suggest that both apply. If you keep only the minutes, you lose the record that makes this a disagreement. And if you look on 16 March, you have another problem. The register still covers the day, while the minutes only establish what the meeting said about the 15th.

A knowledge graph can preserve enough information to answer each of those

questions. Its relationships have named meanings and connect identified things. Its evidence records say who or what made an assertion, where to check it, and under what conditions it applies. A query then selects and interprets those records for a particular purpose. Keeping these steps visible lets a reader challenge an answer and see exactly what a correction would change.

The flow below shows what must survive between a source and an answer. Read it from top to bottom: the arrows describe operations on evidence, while each labelled stage states what the operation produces.

Source: the register and minutes make different statements about version 1.
↓ Retain the wording and its location in each source snapshot.
Mention → entity: link each occurrence of "version 1" to Travel Policy v1.
↓ Preserve the relation, polarity, date, and source together.
Qualified claims: an affirmation from the register and a denial from the minutes.
↓ Select both for Atlas on 15 March and compare their time support.
Answer with evidence: report disagreement and cite both source passages.

The two source statements reach the answer as separate records. Resolving their mentions to the same policy version enables comparison; it gives neither statement authority over the other. Dropping the denial or its date at an earlier stage changes what the final answer can establish.

Northbridge is fictional. You can reproduce the main case from the excerpts, tables, and worked decisions in this primer. Familiarity with tables and identifiers is enough. The primary route starts with the questions Northbridge needs to answer, follows the evidence into the model, and works the policy queries and a correction by hand. The two transfer cases repeat the reasoning for research literature and software dependencies. The alternative route uses a projection of Northbridge to study graph structure. Storage and implementation notes give optional mappings and describe the companion implementations.

Verified against primary sources for the standards and interfaces in Storage and Implementation Reference and the R projection. The case studies and their worked results are synthetic.

2 Design the Graph Around Questions

Start with the Atlas project and the first version of the travel policy. An arrow labelled “applies to” can connect them. A second arrow from Atlas to Northbridge can record that the project belongs to the organisation. The two arrows make it easy to ask which organisation is connected to that policy version through a project. To answer whether the policy governs a particular trip, you would need to know which version, which project, which period, and potentially what sort of trip. The difference between those questions determines how much of the source the model must preserve.

2.1 What an Edge Lets You Ask

A graph has **nodes** and **edges**. In the graph model used here, a node represents an identified thing or a record about one. An edge has a direction and a type. The relationship type defines how to read its endpoints. For `owned_by`, the subject is the policy version and the object is the person named as its owner. Looking up what Morgan Lee owns may involve following the edge in reverse, while the statement expressed by the edge still reads from version to person. An inverse relationship such as “owns” can be defined from the same assertion if the vocabulary guarantees that equivalence. Reversing an arrow and reading the original label would change its meaning.

A **path** is a sequence of edges that you can follow through shared nodes. With version 1 → Atlas → Northbridge, the Atlas identifier is the object of the first edge and the subject of the second. A graph query can bind that shared identifier once and use it in both steps. A relation named `belongs_to` at the second step answers a different question from `sponsored_by`. Even if both edges ended at Northbridge, the resulting paths would establish different things. Define the sequence of relationship types for a question before deciding which connections count.

Consider two systems that both display the same two arrows. One drew them from two unchecked name matches. The other linked an identified version to a project using a dated register entry and the project to an organisation using an identified directory record. If a user wants to dispute the first edge, the systems give very different opportunities for review. The knowledge graph in this primer comprises a graph model and a way to trace its claims, including

the qualifications that the two-arrow drawing omits. Some systems use the term more broadly for a graph with a shared vocabulary. In either usage, the drawing alone gives little information about what the relation means or what supports it.

Graphs are a way to organise and ask questions about data. A two-column list of source and destination IDs with a third column for relationship type can represent directed edges. A path can be followed by matching columns across copies of that list. A service might organise the same records by each node’s outgoing neighbours. Both arrangements can represent the same graph. The distinguishing design work here is choosing stable participants and relationships that readers can combine, and making those relationships precise enough to interpret together. The storage engine is a later decision.

For a single register lookup, a table indexed by project and date may already give the reader everything they need. Consider the work created by separating a policy family, a policy version, a project, an organisation, and a source record. You have to name them consistently and check each relationship. The investment starts to pay when the next questions reuse the distinctions. A project-centred query can combine applicability with ownership. A document-centred query can retrieve all the assertions introduced by one register revision. A comparison can group statements from two sources by the same version and project. Each uses a different starting point and shares the same evidence model.

A graph may also fit poorly when the identifiers have no dependable interpretation. Linking hundreds of pages through a surname gives many paths while making the answers harder to justify. Increasing the number of edges then raises the cost of review. Ask what users will do with a path and how they can check it. A small corpus with a reliable index and clear source locators may meet a discovery need before any entity integration is warranted. At Northbridge, the source roles, the distinct policy versions, and the reviewable same-name case make the extra modelling decisions visible.

2.2 Define the Northbridge Evidence Boundary

A competency question describes an operation the graph must support and what would count as a satisfactory answer. Start with a question someone would use. “Which policy governs Atlas?” is too broad for the supplied records. The analyst can ask what the register records about Atlas on a particular day. A meeting reviewer can ask whether a recorded statement conflicts with the register. These

are useful questions with different admissible evidence. Set the scope before you construct the data, so a convenient query cannot silently expand or shrink the evidence boundary.

The register is the designated record for policy applicability and ownership in this exercise. The directory supports project affiliation and personal identity. The minutes are evidence of what was recorded at the meeting. Each source role is local to a question. The register has no special standing as a record of attendance. The minutes can establish what a meeting stated even when the register is the designated record for policy administration. The exercise compares the evidence and leaves the operational decision about which policy to follow to an accountable owner.

The case uses the following three source snapshots. A **locator** identifies a region within a snapshot. The register locator covers two entries and the minutes locator covers three sentences. Adding a suffix such as `:sentence-2` identifies a single sentence within that region. The table shows the retained internal excerpts. The ordinary view uses the approved excerpts worked in the disclosure test. Dates in the Northbridge example are in 2024 unless a different year is stated.

Source and revision	Locator	Retained excerpt
Authorised policy register, `s-register`, R1	`R1:P1-P2`	“Travel Policy v1 applies to Atlas from 1 January until 1 April. Owner Morgan Lee has directory ID NB-104. Travel Policy v2 applies to Atlas on 1 April.”
Northbridge directory, `s-directory`, D1	`D1:A1`	“Atlas and Orion belong to Northbridge.” D1 also supplies the three directory entries shown in the identity example.

Source and revision	Locator	Retained excerpt
Atlas meeting minutes, `s-minutes`, M1	`M1:paragraph-3`	“On 15 March, version 2 applies to Atlas. On 15 March, version 1 does not apply to Atlas. Alex Kim attended.”

The register and directory snapshots are dated 1 January. Read “until 1 April” as an exclusive end in this case. The register entry about Morgan Lee records the owner of version 1 at the snapshot. No ownership duration is stated. The version 2 register sentence explicitly covers 1 April, and the two minutes sentences explicitly cover 15 March. The date of a meeting alone could refer to when an assertion was made, while what was said might concern a previous week or a future decision. The explicit dates in the sentences are what anchor the applicability assertions.

The project and date boundary matters even when a reader recognises every name. Northbridge and the two projects are in scope. The excerpts give no coverage guarantee for other policies, project activities, amendments, exceptions, or individual trips. A request to check whether a specific Atlas expense was reimbursable would need additional evidence about the expense, the relevant policy terms, and the applicable decision procedure. The appropriate output to that request identifies the missing evidence.

2.3 Turn the Boundary Into Answer Contracts

The five questions below give the model something concrete to support. Each specifies which sources or date matter, and what the user needs back.

Question	Scope and evidence for the answer
Q1. Which version does the register record for Atlas on 1 March?	Examine register applicability for Atlas. Return the version with its assertion, locator, and date.

Question	Scope and evidence for the answer
Q2. Whom does the register name as owner of version 1, and can that ownership be carried forward to 1 March?	Retrieve the version 1 owner record and identify its date. Test the later date separately.
Q3. What do the 15 March minutes record about applicability?	Return both assertions, with their subjects, polarities, dates, and shared locator.
Q4. Do the minutes and register agree about version 1 for Atlas on 15 March, and what does the applicability policy conclude?	Report agreement, disagreement, or insufficient evidence to establish either by comparing applicable assertions from both sources. Separately report the policy outcome using the temporal classifications and cite the evidence for both determinations.
Q5. Which directory person attended under the name Alex Kim?	For an authorised identity reviewer, return a person link only if the evidence distinguishes the candidates. Otherwise return the mention and the reason to abstain.

The answers have different shapes. Q1 asks for a version, while Q3 asks for a set of statements and their evidence. For Q4, agreement requires applicable statements from both sources with the same polarity, and disagreement requires opposing applicable statements. A single source can supply enough evidence for the policy to reach an applicability result while leaving the comparison unresolved. Q2 can answer one part and qualify the other. A design that requires one value for every question would have to suppress some of that information. For instance, it could select the first Alex Kim, choose the register on every conflict, or leave an empty result. The contracts make those outputs distinguishable from satisfactory answers.

Consider Orion. The directory identifies the project, so a query can bind its ID. None of the supplied sources asserts that a version applies to Orion, and none explicitly denies applicability. A search over these sources can return “No ap-

plicability assertion found for Orion in the selected evidence." For an omission to count as negative evidence, you would need a completeness rule for this relation, population, period, and scope. Suppose a designated travel register for 1 March guaranteed that it listed all applicable version–project pairs for every active project and defined omission as exclusion. You would also need evidence that Orion was in that active population. The lookup and absence would then be premises of an inference with a named rule and dated coverage guarantee. The Northbridge excerpts contain neither that guarantee nor the necessary population record.

Source completeness differs from extraction completeness. A perfect extraction of this small corpus would still leave questions about policies outside it. Conversely, even a complete register can yield incomplete answers when an extraction step skips a qualifying entry. Store enough evidence about input coverage and refused passages to distinguish those cases. If the organisation begins to use Q1 to check every project, the model owner should revisit both the question scope and the source guarantees. A high success rate on the Atlas example would tell the owner little about either.

3 Build an Evidence Ledger

The first operation on a source is to retain what it actually contains. A relationship in a document is often embedded in a sentence with an actor, a time reference, or an exception. The ledger keeps those distinctions visible as the sentence is turned into something a query can select. Each stage produces evidence the next stage can use. A review of a wrong answer can then start at the point where two adjacent stages diverge.

3.1 Preserve Mentions and Resolve Entities

A mention is a particular occurrence of a word or phrase in a source. An **entity** is the thing that an occurrence can refer to. A **label** is display text. An **identifier** names an entity within an agreed namespace. For example, a recorded occurrence of "version 1" in the minutes is a mention, Travel Policy v1 is the policy version, and `policy:travel-v1` is its stable local identifier. The same version has a different mention in the register. Changing the display label for the version leaves both source occurrences intact.

Keep the thing you need to query at the right level. The travel policy as a family is the topic of the minutes. The v1 and v2 texts are separate versions, so a relation about one version can change without changing the other. If a system gave the family and its versions one identifier, the two minutes assertions could become a positive and negative statement about the same entity even if they had referred to separate versions throughout. Another case could need an additional revision level, for example if an official version label was kept after an erratum. Version identifiers need to correspond to the distinctions users actually ask about.

The eight local entities in the Northbridge case are listed below. Directory identifiers are matching evidence under the Northbridge issuing authority. The two Alex records denote distinct directory people in this example.

Local ID	Kind	Display label	Directory ID where supplied
`org:northbridge`	Organisation	Northbridge	—
`project:atlas`	Project	Atlas	—
`project:orion`	Project	Orion	—
`policy:travel-v1`	Policy version	Travel Policy v1	—
`policy:travel-v2`	Policy version	Travel Policy v2	—
`person:morgan-lee`	Person	Morgan Lee	NB-104
`person:alex-kim-1`	Person	Alex Kim	NB-201
`person:alex-kim-2`	Person	Alex Kim	NB-202

An identifier often needs three pieces of context to serve as evidence of identity. You need the value, the authority that assigned it, and any interval during which the assignment is valid. Two systems might use NB-104 in separate namespaces. A departed person's account could be reassigned under a different local policy. If those cases are possible, record the assignment history before treating the value as decisive. Here the register and directory refer to the same issuer and compatible dates, so the matching assignment of NB-104 supports linking the Morgan Lee mention to `person:morgan-lee`.

For M1, the case supplies the topic metadata "Atlas travel policy discussion". That supplied context, together with the version numbers, supports linking the shortened "version 1" and "version 2" to the travel-policy versions. The three quoted sentences alone do not establish the policy family. Retain the topic metadata with M1 as part of the link decision; a real import would need to re-

tain the heading, agenda item, or other located context that supplies it. An isolated sentence with the same wording would leave the family unresolved. “Atlas” likewise denotes the project in this supplied context. The model’s type checks would rule out matching an Atlas-named person or application to a project role, but there could still be two projects with the same display name.

The following intermediate records show selected occurrences and the link decisions. Their locators and quoted phrases can be checked in the source table. Repeated phrases such as “Atlas” can be distinguished by their sentence positions.

Occurrence	Source region	Candidate or candidates	Decision and basis
`m-policy-v1`, “Travel Policy v1”	R1, first sentence	`policy:travel-v1`	Link on family and version in the register entry
`m-morgan`, “Morgan Lee”	R1, second sentence	`person:morgan-lee`	Link `r1` on the same directory assignment NB-104
`m-policy-v2-register`, “Travel Policy v2”	R1, third sentence	`policy:travel-v2`	Link on family and version in the register entry
`m-policy-v2-meeting`, “version 2”	M1, first sentence	`policy:travel-v2`	Link using the policy topic and version
`m-policy-v1-meeting`, “version 1”	M1, second sentence	`policy:travel-v1`	Link using the policy topic and version
`m-alex`, “Alex Kim”	M1, third sentence	`person:alex-kim-1`, `person:alex-kim-2`	Abstain in `r2`. Both match the name and the passage supplies no discriminator.

Candidate selection and link acceptance are two distinct operations. To select candidates, you might narrow to people in the relevant directory and compare

names and known aliases. To accept a link, inspect discriminating evidence such as a scoped ID or a corroborated assignment. In Northbridge the candidate selection step returns two candidates for Alex Kim. Neither the order in the directory nor the number of other records about one candidate supplies evidence that the attendee was that person. Record both candidates, the reason the decision stopped, and the review conditions that could change it. Candidate selection can also miss a true match, for example if a name changed and the earlier name is absent from the alias index. A subsequent scoring step cannot recover a match it never received. Dasylva and Goussanou's study of missed candidates in record linkage.

Retaining an unresolved mention lets you state that the source recorded attendance under a name and investigate later without attributing the attendance to a person. The ledger has no attendance relation to either Alex entity. Imagine a system that merges the two people on name and then assigns the attendance to the merged record. Every later query for either person's projects or memberships can now associate those records with the meeting. To repair that error, you would need to split the entity and revisit every dependent link. If you retained the mention and the original directory entries, you can recover those decisions. If the merged entity has become the only record, the evidence needed to undo the assignment may be gone.

3.2 Give Claims and Assertions Separate Roles

The **proposition** at the centre of the dispute is "Travel Policy v1 applies to Atlas". Its key is the ordered combination of subject, predicate, and object. The **claim** gives that proposition the qualifications needed to interpret it, such as the time period and any conditions. An **assertion** is a source-specific presentation of a claim, with polarity, evidence, and admission history. Other vocabularies use "claim" for the entire source record. Here `c1` and `c5` label assertion rows, while their common proposition gives us a way to compare them.

The difference becomes visible when a source quotes another source. If the minutes state "The register says version 1 applies to Atlas", they can attest to what was reported at the meeting. Whether that quotation counts as a separate policy assertion depends on the purpose and wording. Even if it repeats the register accurately, the two statements share one origin. The ledger can preserve the two source occurrences and a derivation or quotation link. An evid-

ence count that treats both as independent would overstate support. Multiple sources can also independently present the same qualified claim, in which case the claim can be shared while the assertions and locators remain separate.

A qualified assertion can be stored as one row with a proposition key, qualifiers, and a source reference. When several pieces of evidence support a row, keep separate assertion-to-evidence links. A single reference field that contains several undifferentiated locators makes it hard to tell which passage supports which detail. For example, “Owner Morgan Lee” supports the relationship in its version 1 entry. The directory record corroborates the identity link. Neither passage independently supplies the whole owner assertion. Keep evidence roles in the chain so an owner correction and an identity correction affect the right decisions.

Define each relation before admitting assertions. The following definitions give participant types and the Northbridge reading.

Relation	Subject → object	Reading within the case
`applies_to`	Policy version → project	The source affirms or denies applicability for the stated scope and period.
`owned_by`	Policy version → person	The register records the person as that version’s owner.
`belongs_to`	Project → organisation	The directory records the project’s organisational affiliation.

A domain or range check uses the participant types. A meaning check requires more than those types. “Morgan reviewed version 1” has a person and a policy version, but switching the endpoints and labelling the result `owned_by` would fabricate an ownership relation. Similarly, a source might say version 1 applies only to domestic trips. If the question asks about all Atlas travel, keep the restriction as a condition and compare it with the requested activity. Dropping the restriction in construction creates an apparent contradiction with a denial about

international travel even when both source statements can hold. The supplied assertions have no trip-class qualifier. That is the limit of their granularity in this exercise.

3.3 Transform Source Wording Into Reviewable Rows

Conservative extraction means proposing assertions from wording you can justify and preserving the reason when you cannot complete a row. Begin with the full minutes sentence “On 15 March, version 1 does not apply to Atlas.” Extract its time phrase, version mention, relation phrase, and project mention in context. Resolve the version and project to the existing IDs. Keep the direction from the version to Atlas. Read “does not” as negating applicability. The source asserts this denial on 15 March and leaves continuation unspecified. Preserve that sentence with the assertion so that the decision can be checked after the wording has been normalised.

The table below works the three main source-derived candidates from spans to admissible assertions. Within each span the participant identity, relation, polarity, and temporal reading must all pass review. The start and end interpretation is developed in the next section.

Retained span	Participant and relation decision	Polarity and time read from the span	Accepted row
R1, first sentence, “Travel Policy v1 applies to Atlas from 1 January until 1 April.”	Version 1 → `applies_to` → Atlas	Affirmation. Inclusive 1 January start and exclusive 1 April end.	`c1`
M1, first sentence, “On 15 March, version 2 applies to Atlas.”	Version 2 → `applies_to` → Atlas	Affirmation. Support for 15 March, later duration unknown.	`c4`

Retained span	Participant and relation decision	Polarity and time read from the span	Accepted row
M1, second sentence, “On 15 March, version 1 does not apply to Atlas.”	Version 1 → `applies_to` → Atlas	Denial. Support for 15 March, later duration unknown.	`c5`

The same process can fail in more than one way. “We discussed whether version 1 does not apply” contains a recognisable negation inside an unresolved discussion. Turning it into a denial confuses a topic with an assertion. “Version 1 does not apply unless approval is granted” contains a condition. The unqualified predicate would be too broad. “Travel Policy v1 applies to Atlas from sometime next spring” does express applicability, while the given words are too imprecise for a day-level start. These are counterexamples to automatic admission in this model. Preserve the span and record respectively that the assertion is unresolved, the condition needs representation, or the start is unsupported at the requested granularity. A different modelling task might use coarser time or retain condition-specific candidate assertions.

The language that carries a decision can span several parts of a document. A table headed “Draft for discussion” can list dates in the same format as an authorised register. A footnote can exclude one project from a row that appears to cover all projects. A scanned date can contain a recognition error. Preserving the extraction evidence means retaining enough context to see these differences, including the source version and the transformation used to obtain the readable text. Matching a string pattern is a candidate-generation technique. Verification compares the candidate’s meaning to the evidence and its context.

A refusal differs from a malformed record. If the source gives an explicit day but the end field precedes the start because a day and month were transposed, inspect the transcription and repair the candidate. If the source itself has incompatible dates, retain the problem for a source owner. If the source has no usable day, an incomplete candidate can remain in a review queue without appearing as an admitted date-qualified assertion. A graph that hides all three as nulls leaves the query unable to decide whether to surface an error, an uncertainty,

or a coverage limitation.

The ledger below is the intermediate representation used for the rest of Northbridge. The other four rows are curated from the supplied excerpts and snapshot context. Their construction evidence is the specified passage and interpretation. In the examples an optional mention can be absent, as for `c7`, while a source and a locator are required for each admitted assertion.

ID	Subject → predicate → object	Polarity	Source	Valid-time support
`c1`	Version 1 → `applies_to` → Atlas	+	R1	1 January to 1 April, exclusive end
`c2`	Version 1 → `owned_by` → Morgan Lee	+	R1	1 January, unknown end
`c3`	Atlas → `belongs_to` → Northbridge	+	D1	1 January, unknown end
`c4`	Version 2 → `applies_to` → Atlas	+	M1	15 March, unknown end
`c5`	Version 1 → `applies_to` → Atlas	−	M1	15 March, unknown end
`c6`	Version 2 → `applies_to` → Atlas	+	R1	1 April, unknown end
`c7`	Orion → `belongs_to` → Northbridge	+	D1	1 January, unknown end

“Version 1” and “Atlas” are readable labels for the stable IDs above. A graph operator should match IDs. If a second Atlas project were introduced, its label could look identical while its identifier and affiliations differed. Likewise the assertion ID differs from the ID of every participant. A review or correction can

point to the particular row it concerns. This is how the ledger allows `c1` and `c5` to coexist and be retrieved as evidence about the same proposition without fusing their polarities or dates.

3.4 Follow the Evidence Back to Its Source

Provenance is the history and origin of a record. In a query result it connects a conclusion to evidence a reviewer can inspect. In a rebuild it connects a changed source to the assertions and derived results that depend on it. For `c5`, a reviewer follows the assertion ID to M1 and `M1:paragraph-3`, locates the second sentence and its "version 1" mention, checks the entity decision and the scope of "does not", and recovers the asserted day from the sentence. If a later result omits the denial, those records distinguish a lost source, a missing candidate, an inadmissible row, an incorrect query, and a lost citation.

A source name such as "Atlas minutes" is insufficient when the document changes. Retain a version key and a reproducible locator, such as a page and paragraph in a snapshot or a stable row key and column. For a character range, identify the version of the extracted text as well as the underlying source. Normalising whitespace or changing the text extractor can move every following offset. A content digest can help check that the reviewed snapshot is the one processed. It cannot recover deleted material or establish whether the sentence was authoritative.

Record the source publisher, the speaker or author where relevant, the extraction activity, and the reviewer according to their actual roles. A corrected import can come from the same publisher under a new source revision. A new interpretation of the old minutes can come from a changed extraction method or review decision while the bytes stay the same. Those two corrections can produce the same answer change and still require distinct histories. A reviewer should be able to see which one occurred. Provenance about how a record was produced is also distinct from the scope of the record and the authority assigned to it for a question.

For a hand-worked answer you can cite a locator and the sentence directly. At larger scale, keep an explicit evidence reference in the result even if users normally see a short summary. An answer that returns `c5` plus the locator lets a reviewer find the original paragraph. If the source link was lost, the answer should surface an evidence-integrity problem. A guessed citation from the sub-

ject label could attach the register to the minutes denial, leaving the user with a convincing citation that cannot support the statement. This is a failure of the evidence chain even if the displayed conclusion happens to match what the sources say.

4 Interpret Time, Conflict, and Uncertainty

The ledger contains both an affirmation and a denial of version 1 applicability. To say whether they conflict for a question, first ask whether the two assertions concern the same proposition and scope, then whether their supported time overlaps on the requested day. The time decision belongs to each row. After classification, an answer policy decides what the selected rows collectively establish.

4.1 Distinguish Supported Periods From Unknown Duration

The time convention uses calendar days and an inclusive start. A **bounded** period ends at a stated exclusive date. **Unbounded** means the source explicitly asserts continuation, such as “from 1 January and until further notice”. The statement remains applicable from its start in the selected release, subject to new evidence or correction. An **unknown** end has an inclusive last supported day, or **support horizon**. After that horizon the row is temporally indeterminate. A statement tied to one day supports that day. In this model it does not establish that the relation ended on the following day, so the one-day evidence uses an unknown end rather than an inferred exclusive end.

Date language needs a convention tied to the source. “Until 1 April” is exclusive by assumption in Northbridge. A source that says “through 1 April” could mean that 1 April is included, and its half-open representation would end on 2 April. If the date refers to a timestamp, first settle the time zone and level of precision relevant to the question. A change at midnight can fall on different calendar days depending on that convention. A register with month-only precision calls for a policy about month-level evidence, or for withholding a day-level conclusion where the month alone is insufficient. Inventing a more precise timestamp conceals that uncertainty.

Given an already validated row and a date, compare the date with the start first. If it precedes the start, the row is outside. For a bounded end, a date at or after

the end is outside and a date before it is applicable. For explicit continuation the row is applicable on or after the start. For an unknown end, a date through the horizon is applicable and a later one is indeterminate. Here *applicable* means the assertion can contribute its polarity to the decision at that date. A negative assertion can therefore be applicable while its content is a denial of the policy's applicability. Keep the two uses separate when explaining results.

For `c1`, 31 March is within the interval and 1 April is outside. For `c5`, 14 March is before the start, 15 March is within the support horizon, and 16 March is beyond it. If you entered both ends as blank with no end kind, one query could treat them as permanent and another could reject both. An explicit end kind allows a validator to catch contradictory combinations. A bounded end must come after its start. An unknown end needs a horizon on or after the start. An unbounded end needs evidence of continuation. A blank end cannot supply that evidence.

Suppose a separate bulletin says "Version 1 applies to Atlas on 15 March and will continue to apply until further notice." A new assertion with that explicit continuing period would be applicable on 16 March in a release that admits it. This example differs from extending `c1` or `c5`. You would retain the bulletin and its source scope as a new assertion and evaluate it alongside other admissible evidence. If a subsequent bulletin actually terminates the instruction, that gives evidence for an end or a supersession event. An unknown end still carries no answer about whether the underlying condition in fact continued.

4.2 Keep Domain Time and Recorded Time Apart

Valid time describes when the source says a relationship holds, or the period its evidence supports. **Recorded time** concerns when an assertion or revision entered the graph. The source publication date is a third useful piece of provenance. Suppose the register publishes a correction on 20 April, the reviewer sees it on 22 April, and a graph release admits it on 25 April. A question about March under that release can use the revised interval. A question about what the graph returned on 21 April needs the earlier release, even though the correction had already been published. A question about when a human reader could have encountered it needs a source access history and publication context.

Consider a user who reports "Your answer for 15 March changed after I saved it." To reproduce the old answer you need the original question parameters, the release selected then, the evidence and policies it admitted, and the requester's

view of the data. Querying a current release with the valid date set to 15 March may instead reproduce the revised answer. Both questions can have sensible, different results. The original excerpts supply valid-time support and synthetic source dates. They do not supply an operational history of when anyone first knew a statement. The lab supplies explicit teaching releases so that you can compare answer behaviour without inventing that history.

4.3 Recognise Disagreement at the Right Grain

The direct contradiction in Northbridge is between `c1` and `c5` on 15 March. Both concern the same version, relation, project, and day. Their polarities oppose each other. The version 2 affirmation `c4` adds context. It would be incompatible with a version 1 affirmation only if an explicit exclusivity rule said that at most one version could apply in that same scope. Real policy terms might permit parallel applicability for different activities or a transition period. A matching cardinality rule would have to express those conditions. Counting two version nodes linked to Atlas is insufficient to diagnose a contradiction.

A negative assertion is evidence with its own source. It never needs to be represented by deleting a positive relationship. A missing applicability edge for Orion is a third situation. It supplies neither affirmation nor denial. There can also be an excluded assertion whose text is available but whose scope falls outside the query. For instance, an external organisation’s travel policy could mention its own Atlas project, with no authority over Northbridge. Selecting by the entity and source boundary prevents that row from entering the comparison. Keeping the reason for exclusion can help when a user wonders why a seemingly relevant search result was omitted.

Different uncertainties call for different responses. The two Alex candidates expose identity uncertainty. A request involving an unsupported start date exposes uncertainty in temporal scope or granularity. The version 1 disagreement is present even with an unambiguous extraction. A corpus missing Orion policy records presents a coverage issue. In each case a single “low confidence” value hides the next useful step. Seek a distinguishing identifier, a dated source, a reconciliation by an accountable source owner, or additional coverage, respectively. Record uncertain qualifications in a form the query can use. Reserve a numerical score for a clearly defined and tested quantity, such as the reliability of a particular candidate-ranking procedure under a stated sample, rather than

using one number to represent all of these conditions.

Northbridge's cross-source policy gives priority to a demonstrated conflict between applicable rows. Without that conflict, any temporally indeterminate selected row leads to abstention for temporal scope. With neither of those conditions, an applicable positive yields "supported", an applicable negative yields "refuted", and the absence of applicable rows yields abstention for lack of evidence. The policy is deliberately cautious. It can withhold a result even if the indeterminate evidence has the same polarity as the applicable evidence. An organisation might choose a different rule, such as restricting a register question to the designated source or determining whether only an indeterminate opposing polarity should block a conclusion. Changing this rule changes the answer and needs its own documented version and evaluation.

Work the consequence with version 2 on 1 April. Select its Atlas applicability assertions from both sources: `c4` from the minutes and `c6` from the register. `c4` is indeterminate because its support stops at 15 March; `c6` is applicable on its stated 1 April day. Both are positive, so there is no applicable disagreement, yet the conservative policy abstains because a selected row is indeterminate. Under an alternative rule that blocks support only for an indeterminate opposing polarity, `c6` would instead yield "supported", citing `R1:P1-P2`, sentence 3. That rule would still report the minutes as indeterminate, and could not claim that both sources agree on 1 April. A register-only question also yields version 2, but reaches it by selecting a different evidence boundary. Keep those two policy choices distinct when comparing answers.

5 Answer Questions From the Graph

The query procedure binds the entities and relation types, selects sources and the release, classifies valid time when the question requires it, and explains the result using evidence references. All of this can be done with the seven ledger rows. A **binding** gives a value to a role in a pattern. The unbound roles are the values you want to find. Multiple patterns can share one binding, for example when the object of the first relationship must be the subject of the next.

5.1 Look Up a Version and an Owner

Start Q1 by binding the object to `project:atlas` and the relation to `applies_to`. Leave the subject open. The first selection returns `c1`, `c4`, `c5`, and `c6`. For this question the source boundary admits R1, leaving `c1` and `c6`. It matters that you selected the register because the full set includes a denial as well as a second source's version 2 assertion. The selected rows are then classified individually.

The register candidates produce the following intermediate results. The first column is the tested day and the last column gives the answer under the register scope.

Day	`c1`, version 1	`c6`, version 2	Answer
1 March	Applicable, positive	Outside before its start	Version 1, `c1`
31 March	Applicable, positive	Outside before its start	Version 1, `c1`
1 April	Outside at its exclusive end	Applicable, positive	Version 2, `c6`
2 April	Outside after its end	Indeterminate beyond its horizon	Abstain on temporal scope

A complete Q1 answer is "The register records Travel Policy v1 for Atlas on 1 March, in `c1` at `R1:P1-P2`." It can show the tested dates of both candidates so a reviewer sees why version 2 was excluded. The register's 1 April entry supports version 2 on that day. Nothing in the excerpt fixes the end of version 2's applicability or supplies a later supported day. This is why the first two rows use the bounded interval for version 1 and the final row abstains. Selecting the latest version label regardless of date would give the wrong answer in March and overstate what is known on 2 April.

For Q2 fix the **subject** to `policy:travel-v1`, the predicate to `owned_by`, and the source to the register. Leave the **object** open. The only matching row is `c2`, whose object is `person:morgan-lee`. Binding the object to version 1 would ask for a policy version in the person role and return no matching row. A type-aware pattern can flag that mistaken query before looking for evidence. Querying the correct direction gives two parts to the answer. R1 names Morgan Lee as owner

at its 1 January snapshot. Continued ownership on 1 March is indeterminate because the tested date is later than the 1 January support horizon.

For ownership, inspect both the relationship and the identity evidence. The owner sentence is in the version 1 entry and `r1` ties the mention to the Morgan Lee entity using NB-104. The registration of that identifier supplies the identity link. It carries no additional valid time for the ownership assertion. Even if Morgan Lee is still in the directory on 1 March, employment or directory presence would need a separate rule to establish policy ownership. Q2 relies on `c2` at `R1:P1-P2` and gives the date limits alongside the name.

The answer shape also distinguishes ambiguity from multiple legitimate results. If the register named joint owners, the model would need to preserve both and tell the user the role can have more than one holder. A one-owner constraint would instead surface a conflict or modelling error. Northbridge has one relevant owner assertion, so it does not decide the organisation's general ownership cardinality. A query should not introduce such a rule by taking the first row when it encounters a second.

5.2 Inspect Assertions and Their Evidence

For Q3 bind the source to M1, the predicate to `applies_to`, and the object to Atlas. Keep both polarities. This gives `c4` and `c5`. The first reports that version 2 applies on 15 March and the second denies version 1 for the same day. To answer what the minutes record, cite both at `M1:paragraph-3` and keep their distinct sentences. The positive version 2 result and the negative version 1 result belong together in the output. A summary that says only "The meeting discussed version 2" would fail to answer the question even though it names a relevant subject.

A provenance query can start from either end. Starting with the version 1 proposition, retrieve `c1` and `c5`, find their source snapshots, and use the locators and mentions to identify their passages. Starting with M1, retrieve all assertions derived from it and review the list when a source change arrives. This reverse lookup is valuable when one corrected source contains several claims. The ledger describes where each assertion came from. It can therefore return the full source evidence for `c7` from D1 even without an optional Orion mention record. Requiring a mention for every retrieval would drop `c7` and make a populated ledger look incomplete. A missing required source reference is a different

condition and should raise an integrity issue.

Q5 uses the identity decision as its answer unit. An authorised reviewer binds the occurrence `m-alex` in M1 and retrieves `r2`, its two candidate people, and the reasons why no candidate was selected. The result is an unresolved mention with a locator. This retains the fact that the minutes record attendance under that name. It supplies no accepted person-to-meeting edge. Starting from a directory person and asking what meetings they attended can return confirmed links if the graph has them, while the unresolved occurrence stays visible only through a suitable authorised review route. The governance section works the answer for a different audience.

5.3 Compare the Same Proposition Across Sources

For Q4 fix both endpoints to version 1 and Atlas and the predicate to `applies_to`. Admit R1 and M1. The match returns `c1` and `c5`. Version 2 falls outside the subject binding. The owner assertion falls outside the predicate binding. Each surviving row has its own temporal fields, so classify the same query day against each. On 15 March `c1` covers the day because it is before 1 April and after 1 January. `c5` also covers the day because the start and inclusive horizon are both 15 March. The two applicable rows have opposite polarities, so the sources disagree. The policy takes the conflict branch and abstains on applicability.

Changing the requested date gives these classifications under the same evidence selection.

Day	`c1` (+)	`c5` (−)	Source comparison	Applicability policy result
31 December 2023	Outside	Outside	Neither source has an applicable assertion	Abstain for lack of evidence
1 March	Applicable	Outside	Register alone has an applicable assertion	Supported

Day	`c1` (+)	`c5` (−)	Source comparison	Applicability policy result
15 March	Applicable	Applicable	Disagreement	Abstain on conflicting evidence
16 March	Applicable	Indeterminate	Register alone has an applicable assertion	Abstain on temporal scope
1 April	Outside	Indeterminate	Neither source has an applicable assertion	Abstain on temporal scope

On 16 March the result still withholds a conclusion, and the reason has changed. There is only one applicable polarity. The minutes supply no denial applicable to that day under the chosen horizon, and their unknown duration keeps them relevant as indeterminate evidence under the conservative policy. Extending the denial as a definite negative would falsely preserve the 15 March conflict. Dropping the indeterminate row would give a definite positive under a different answer rule. The query needs to preserve the classification before aggregation so a reviewer can tell which policy was used.

In the original Q4 answer, tell the reader "The register and minutes disagree about Travel Policy v1 for Atlas on 15 March. `c1` at `R1:P1-P2:sentence-1` affirms it and `c5` at `M1:paragraph-3:sentence-2` denies it. Applicability is unresolved under the cross-source policy." A register-only answer on 15 March would report `c1` as applicable and would need to say explicitly that the question was about the register. A service that chose a governing answer using a source priority would need authority for that decision and a way for permitted readers to see the contrary evidence. The comparison here establishes a disagreement in the selected records and gives the source owner something specific to investigate.

5.4 Follow a Bounded Path

Suppose the question is "Which organisation is associated with the project reached from version 1 via the register and directory?" Bind the start to

`policy:travel-v1`. From R1 follow a positive `applies_to` assertion. The matching `c1` binds the intermediate node to `project:atlas`. Next find a positive `belongs_to` assertion in D1 with that project as subject. `c3` matches and binds the final node to `org:northbridge`. `c7` has the same final node but its subject is `project:orion`. It fails the shared Atlas binding. Return version 1 → Atlas → Northbridge with `c1` and `c3` and their locators.

The source and polarity selection are part of this particular path question. Using `c5` as a positive hop would reverse the meaning of the evidence, while allowing the `owned_by` predicate would change the question. If you ask the same pattern for more than one path, retain the evidence for each route. Two assertions about the first hop and three about the second can generate six evidence pairs for a single path of nodes. Whether to count one path or six depends on whether the task counts distinct node sequences or evidence combinations. A grouped answer can display one path and list the evidence behind it. It should preserve the combinations that are actually admissible.

For a dated version, classify both hops at the same date. On 1 January `c1` and `c3` are applicable. On 1 March `c1` is applicable and `c3` is indeterminate, so the path is indeterminate. On 1 April `c1` is outside. An outside required hop makes the whole path outside, even when another hop is indeterminate. In effect the path requires the intersection of the periods for its premises. Support for each at some point in the record does not establish that their periods overlap. A dated association rule based on this path could derive support only where both premises support the same day.

The path still leaves the policy and organisation connected through a particular project. Extending applicability to Orion would require a separate relation or rule, such as an explicit organisation-wide policy statement and evidence that Orion was in its scope. The available evidence contains neither. A user can follow the path backwards for exploration if the question allows inverse traversal, but the evidence then still has its original directions. An undirected drawing might connect version 1 and Orion via Northbridge. That connection supplies no legitimate inference of policy applicability to Orion. This difference between navigational reachability and the meaning of a typed, evidence-backed path also governs the alternative projection.

Carry these query results into the correction lab as the baseline. Preserve an answer record naming the question, date, source versions, graph release, audi-

ence view, selected rows, temporal classifications, source comparison, policy branch, result, and displayed evidence locators. The record makes a changed answer comparable with its predecessor. A release name alone does not identify which audience view produced it.

The dated path is also a dependant of `c1`, even though it does not decide Q4. Record that dependency before changing the assertion: a correction can affect paths, counts, explanations, caches, and projections differently. The next lab changes one input or stage at a time and uses the classifications already worked here to locate the consequence.

6 Diagnose and Correct the Answer

The Q4 comparison supplies the lab's baseline: on 15 March, applicable `c1` and `c5` give opposing evidence, and the policy abstains. Keep its question, source boundary, date, and policy fixed through three controlled passes. An extraction repair restores a statement from the same source; a source correction changes the evidence to be represented. The following flow shows where those two interventions differ and how either reaches a release check.

Baseline: retain the Q4 inputs, selected rows, classifications, and citations.
↓ Diagnose the first stage where the observed record diverges.
Diagnosis: was the minutes denial lost, or did the register wording change?
↓ Repair extraction from M1, or admit the corrected register as R2.
Revised ledger: retain the earlier derivations and identify the new evidence.
↓ Apply the audience contract, then evaluate the permitted answers.
Release regression: compare changed dates, results, paths, and citations.

The branch at diagnosis matters. Restoring a lost denial should recreate the original disagreement. Admitting a corrected interval can legitimately remove it. The final regression check must establish which happened, including cases where the answer text stays the same but its evidence changes.

6.1 Trace a Failure Through Stages

Diagnose in order of information loss. Acquisition obtains a specific source revision. Candidate extraction identifies a possible statement and its span. Verification checks that the candidate means what the source says in context. Asser-

tion admission gives the verified candidate an identifier, participants, polarity, time scope, source, and locator. Selection retrieves rows matching the question. Temporal classification decides whether each selected row is applicable, outside, or indeterminate. Inference or answer policy combines the classified rows. Disclosure transforms the authorised result into the result for a particular audience. Presentation then emits text, citations, paths, counts, and cached forms.

Validation can reject duplicate IDs, unknown entities, bad polarity values, missing sources, incoherent dates, and broken joins. It cannot establish correct negation, current directory evidence, or complete answer display. Verification must return to the span.

Use the smallest discriminator that separates causes. A 15 March `supported` result may arise because extraction dropped `c5`, selection filtered negatives, classification put the minutes outside, policy ignored conflict, or presentation lost a citation. Inspect the span, candidates, assertions, selection, classifications, policy trace, then rendered result.

Work three representative failures in that order before using the compact reference below. Each starts from an observable symptom and chooses a test that can distinguish the cause. Applying the same repair to every plausible cause could make one answer look correct while leaving the next question broken.

A denial disappears before admission. The symptom is a Q4 answer of `supported` on 15 March showing only `c1`. Inspect the retained `M1:paragraph-3` span, then its candidate log. The source contains “version 1 does not apply”, but the log has no candidate for that sentence. This locates the first loss at candidate extraction. If `c5` were already admitted, the same symptom would instead require a selection test.

The change is to admit the negative candidate with version 1 as subject, Atlas as object, 15 March as its supported day, and the minutes sentence as evidence. Re-run the unchanged Q4 selection and require exactly `c1` and `c5` before evaluating policy. Both are applicable on that day, so the repaired answer reports disagreement and abstains. This checks restoration of the omitted evidence; it does not settle which policy version should govern.

A retained denial is classified on the wrong day. Suppose both rows reach the query, but `c5` is marked outside on 15 March. Compare the sentence’s as-

serted day with the stored start and support horizon. A shifted date, a truncated horizon, or substitution of publication time for the asserted day would explain this symptom. If both stored dates are 15 March, test the classifier's treatment of the inclusive horizon before changing the row.

Restore the source-supported date if transcription was wrong, or repair the classifier if the row was already correct. The discriminating boundary test requires `c5` to be applicable on 15 March and indeterminate on 16 March. Extending the denial indefinitely would make the central date pass while creating a false conflict on the next day. A regression must check the neighbour as well as the reported failure.

The comparison survives, but its evidence is lost in presentation. Suppose the trace contains applicable `c1` and `c5` and reaches the conflict branch, while the displayed answer says "The records disagree" and cites only the register. Compare the decisive assertion IDs with the emitted evidence links. The inference is intact; citation assembly has discarded the negative premise. Re-extracting the minutes cannot repair that later loss.

Emit an authorised locator for each decisive row. The internal replay needs `R1:P1-P2` and `M1:paragraph-3`; the ordinary view will use the approved sentence-level excerpts established in the disclosure section. Check the links as well as their labels, so each resolves to evidence that supports its assertion. The answer is incomplete if a reader can inspect only one side of the claimed disagreement.

The remaining cases use the same symptom → discriminator → change progression. The table is a reference for choosing the next hand check, with the expected result stated alongside the repair.

Failure and stage	Discriminating check	Change and regression result
Q4 omits an admitted `c5`: selection	Inspect matches before polarity and source filters.	Admit both sources and polarities; original Q4 retrieves exactly `c1` and `c5`.
Q4 retrieves both applicable rows but says `supported`: policy	Inspect branch order with both polarities.	Handle conflict first; opposing applicable rows give disagreement and abstention.

Failure and stage	Discriminating check	Change and regression result
Q1 returns version 1 on 1 April: time boundary	Check `c1` against its exclusive end.	Treat the end as outside; 31 March is applicable and 1 April is outside.
The 1 March path is called supported: path evaluation	Classify each hop independently.	Keep `c1` applicable and `c3` indeterminate; the path is indeterminate.
Version 1 is said to apply to Orion: join or rule	Inspect shared IDs and the asserted propagation rule.	Join `c1` to `c3` through Atlas; `c7` supplies no Orion applicability.
Q5 identifies an Alex Kim for an ordinary user: resolution or disclosure	Check `r2`, its evidence, and the audience view.	Keep `m-alex` unresolved with two candidates; require the fixed ordinary response in both restricted-record states.
An ordinary count changes from zero to two: aggregate disclosure	Compare paths, counts, search, exports, and cache views.	Filter before traversal and separate caches by audience; paired ordinary outputs must match.
A revised release serves an old cached Q1 answer: propagation	Compare release, source, derivation, and cache dependencies.	Recompute or invalidate `c1` dependants; the revised release cannot serve the original derivation.

6.2 Run the Three Controlled Passes

The correction experiment keeps Q4, its source boundary, the queried date, and the conservative policy fixed. Change one stage per pass. The baseline and Pass A use the original register statement, ending version 1 immediately before 1 April. Pass B repairs only the positive-only candidate filter. Pass C is a hypothetical source correction that changes the register wording to end immediately before 1 March. `c4` and `c5` always remain anchored to the original minutes and

do not acquire evidence after 15 March.

In Pass A, retain `M1:paragraph-3` and record the extraction method, then use a candidate filter that drops the negative statement before admission. The ledger therefore contains `c1` but not `c5`. Q4 retrieves only one applicable positive assertion. The policy returns `supported`. This output passes neither the pre-registered evidence boundary nor the human comparison with the retained span. Its apparent clarity is the symptom, not a successful resolution.

In Pass B, use the exact same minutes evidence to admit `c5`. Do not rewrite the source, expand its date range, or add an authority rule. Verify the phrase “does not apply” as negation of the applicability predicate, verify Atlas and version 1 in the sentence, and retain `M1:paragraph-3` as the locator. Q4 now retrieves `c1` and `c5`, classifies both as applicable on 15 March, and reaches the conflict branch. The output says that the records disagree and withholds the applicability conclusion. The repaired extraction does not prove which source should govern. It restores evidence that the earlier answer wrongly omitted.

In Pass C, revise only the hypothetical register source. At `R2:P1-P2`, replace the first sentence with “Travel Policy v1 applies to Atlas from 1 January until 1 March.” Keep the exclusive-end convention and carry over the other two sentences and the original 1 January snapshot context. Rebuild `c1` and verify both unchanged spans against R2 before carrying forward the values of `c2` and `c6`. Each row retains its assertion ID and gets an R2 derivation in the revised release `G-C`. Keep the R1 derivations for replay of the earlier releases.

The three register derivations for `G-C` are as follows.

Assertion in `G-C`	R2 evidence	Derivation and retained support
`c1@G-C`	`R2:P1-P2`, sentence 1	Rebuild the version 1 affirmation with inclusive start 1 January and exclusive end 1 March.

Assertion in G-C	R2 evidence	Derivation and retained support
c2@G-C	R2:P1-P2, sentence 2, “Owner Morgan Lee has directory ID NB-104.”	Carry forward the owner assertion and confirm r1 against the same NB-104 assignment in D1. The ownership support horizon remains 1 January.
c6@G-C	R2:P1-P2, sentence 3, “Travel Policy v2 applies to Atlas on 1 April.”	Carry forward the version 2 affirmation with start and support horizon of 1 April and unknown end.

On 15 March, revised c1 is outside and the register has no applicable version 1 assertion. The sources have no applicable disagreement and there is no evidence of agreement. The minutes denial c5 alone yields refuted under the policy. Cite R2:P1-P2 for the revised interval and M1:paragraph-3 for the denial. The result concerns the evidence in the revised release.

The unchanged owner and version 2 wording also needs a new evidence chain. Their values can stay the same while their accepted locators move from R1 to R2. The final release regression gathers these carry-forwards with the changed answers and boundary dates; compare both values and derivations there.

The version 1 to Atlas to Northbridge path illustrates a correction’s downstream effect. Under the original source, the undated positive projection contains the version 1 to Atlas edge. Under C it still contains that edge if the projection discards valid-time qualifiers. A dated projection for 29 February may retain it. A dated projection for 1 March and 15 March must exclude it because revised c1 is outside. The 1 January path is still supported because both hops cover that day. The 1 March path becomes outside because the revised first hop is outside, even though the second hop is indeterminate. Retain both classifications in the trace.

Do not repair C by changing a date in place. Preserve the original source ver-

sion where permitted and record the new source version, locator, admission, reviewer or authorised process, reason, rebuilt values, and affected releases. `c1` supports comparison but does not license overwriting prior evidence.

7 Govern Disclosure and Approve a Release

The identity and release controls begin with Q5 because a correct internal result can still be an unauthorised external disclosure. `m-alex` is a mention in `M1:paragraph-3`, not a person. The directory has two Alex Kim candidates. The minutes do not supply a directory identifier or another authorised discriminator. Resolution decision `r2` therefore records both candidates and abstains from linking the attendance mention to either person. The absence of a person edge is a decision about evidence sufficiency. It is not evidence that neither candidate attended.

7.1 Separate Authorised Review From Ordinary Use

Apply purpose and access constraints before a traversal, path test, count, search operation, export, or cache lookup. An ordinary project user may receive policy applicability and approved ownership evidence. An authorised identity reviewer may inspect the restricted attendance span, `m-alex`, the candidate records, and `r2`. The distinction is about a permitted purpose as well as a role. An authorised reviewer investigating resolution quality may see material that an ordinary user with the same curiosity may not.

The following boundary makes the lab's answer contract concrete. It must be implemented in the data view and query plan, not merely in a front-end label.

Request or artefact	Ordinary project user	Authorised identity reviewer
Q2 owner lookup	Morgan Lee as the register's recorded owner on 1 January, with an approved register excerpt that removes `NB-104`	The same result and the evidence needed to check the register-to-directory linkage where that review is authorised

Request or artefact	Ordinary project user	Authorised identity reviewer
Q4 applicability comparison	The two assertions with approved excerpts `R1:P1-P2:sentence-1` and `M1:paragraph-3:sentence-2`	The same policy result, plus restricted originals only when needed for an authorised review
Q5 identity question	"Attendance identity is unavailable in this view." Return this exact bounded status whether restricted material exists or not	`m-alex`, `M1:paragraph-3:sentence-3`, both candidates, `r2`, and the reason no link was admitted
Attendance-candidate paths, counts, search facets, exports, and caches	No result derived from restricted mention or candidate edges	Available only within the authorised view and its controlled export rules

The ordinary Q5 answer must not distinguish "no attendance record exists", "a restricted mention has no candidates", and "a restricted mention has two candidates". A more informative ordinary answer lets a requester compare releases, query names, or combine counts with other data to infer protected facts. The bounded wording deliberately leaves those states indistinguishable. It protects the fact that there is a restricted identity-resolution case as well as the identities in it.

Work a paired disclosure test using the original R1 and M1 policy assertions. In the first release candidate, admit the restricted attendance evidence, `m-alex`, its two candidate links, and `r2`. In the second, omit those restricted records from the release. Hold the source revisions, directory, policy evidence, and approved excerpts fixed. For the ordinary view, publish `M1:paragraph-3:sentence-2` as a standalone excerpt that resolves to exactly "On 15 March, version 1 does not apply to Atlas." The ordinary lookup serves that text from the approved evidence view. The paragraph locator and adjacent sentences are available only through separately authorised lookups. The register citation likewise resolves to the approved first sentence.

Filter out the restricted records before evaluating the requests. The table gives the ordinary outputs for each release candidate. The count is the number of visible attendance-candidate links in this view.

Ordinary request	Release with restricted records admitted	Release with restricted records omitted
Q4 on 15 March	Sources disagree. Abstain on applicability. Cite `R1:P1-P2:sentence-1` and `M1:paragraph-3:sentence-2`.	Sources disagree. Abstain on applicability. Cite `R1:P1-P2:sentence-1` and `M1:paragraph-3:sentence-2`.
Q4 minutes citation lookup	“On 15 March, version 1 does not apply to Atlas.”	“On 15 March, version 1 does not apply to Atlas.”
Q5	“Attendance identity is unavailable in this view.”	“Attendance identity is unavailable in this view.”
Visible attendance-candidate count	0	0

In the authorised identity view, the first release candidate has one unresolved occurrence with two candidate links. In the second release candidate that occurrence and its links are absent. The ordinary results in the table follow from selecting the same approved evidence and the same graph without attendance-candidate links in both releases. Apply the same paired comparison to path existence from Atlas or the meeting to people, path counts, name searches, type counts, exports, and cache retrieval.

First select the audience-authorised graph and evidence. Then evaluate paths, counts, and aggregates. Computing two full-graph candidate paths and hiding labels already leaks a count. Partition storage, materialisations, indexes, and caches by view identity and access context.

One mention with two candidate links can produce two restricted paths, not two attendees. The ordinary count must not change between paired releases. Test overlapping aggregates because subtraction, release comparisons, and repeated filters can reveal a suppressed fact. A deployment needs a threat model for the observable combinations.

An authorised future source may provide a legitimate discriminator for `m-alex`, such as a directory identifier with a compatible issuer, date, and permitted purpose. Create a new resolution decision. Link it to the original mention, new source version, review basis, and authorisation. Rebuild authorised dependants. The ordinary view remains subject to its own disclosure contract. It need not expose that a restricted resolution changed, and it must not reveal the changed number of candidate paths.

7.2 Version Releases as Evidence Products

Give releases identities that separate source versions from graph outputs. A source snapshot can have a stable source identity and revision. An assertion can retain stable shorthand `c1` while its derivation refers to a particular source revision. Use `G-A`, `G-B`, and `G-C` as the separate graph release identities for passes A, B, and C. Where it is helpful to disambiguate the assertion representation inside a release, write `c1@G-C`. The shorthand identifies the assertion as represented in the graph release that uses the corrected register.

Both `G-A` and `G-B` use register snapshot R1 and minutes snapshot M1. They differ in extraction coverage. `c1@G-B` starts on 1 January and has an exclusive end of 1 April. `G-C` selects R2, M1, and D1. Its register assertions `c1@G-C`, `c2@G-C`, and `c6@G-C` point to the three R2 spans in Pass C. Its minutes assertions `c4` and `c5` still point to M1. State source revision identities in release metadata. Avoid labels like “latest register”.

Every release needs an evidence manifest. Record admitted source versions and locators, namespace and vocabulary versions, extraction, verification, resolution, temporal, answer, and access policies, derivations, validation, evaluation-item revisions, known gaps, approver, timestamp, and published audience views. The manifest establishes what must be recomputed after correction or withdrawal.

Before publishing `G-C`, compute an impact set from the register revision and its three derivations. For `c1` it includes Q1 and Q4 answers, dated version 1 to Atlas to Northbridge paths, dated projections and their search indexes, evaluation items tied to the original source, and downstream inferences whose premises include `c1`. For `c2` and `c6` it includes ownership and version 2 answers whose conclusions can remain the same while their evidence changes. Include explanations, citations, exports, and caches for each affected product.

Build these products only after the audience views are defined. Their evaluation must judge what that audience is permitted to receive, and the final release check must compare the resulting values and evidence with the previous release.

Distinguish three events in the release record. A domain end of validity says the proposition ceased to hold at the end of its stated interval. A correction says an earlier evidence representation or interpretation was wrong or superseded, as with the hypothetical change from 1 April to 1 March. A retraction withdraws reliance on a source or assertion, perhaps because it was unauthorised, fraudulent, or no longer retainable. An expiry can occur without a correction. A correction can preserve the source history. A retraction may make a prior answer non-reproducible from retained content while preserving a permitted withdrawal notice.

Correction handling starts with provenance, not a manual answer edit. Locate every assertion derived from the source revision. Build the candidate replacement from the corrected source. Verify its span and temporal semantics. Create or update the derivation record. Recompute affected answer products and evaluate the regression table. Invalidate materialisations, indexes, caches, exports, and reports that carry affected content. Publish the new release and its manifest together. Keep an approved route for users to report a result using the release identity, question, and evidence IDs.

Retraction handling first stops new use, then applies deletion, restricted-retention, or tombstone rules across evidence stores, views, replicas, indexes, caches, exports, and extracts. Re-run disclosure tests where removal changes counts or releases. A tombstone can explain non-reproducibility without publishing removed content.

Provenance can create risk. Locators, reviewer identities, and release differences can disclose protected facts. Apply audience policy to manifests, derivations, notices, and audit trails. Do not expose stable local identifiers merely because they are not names.

Release approval needs named responsibility. Source admission owners decide whether a source version may enter a release. Domain owners define relationship meaning and any authority rule. Identity reviewers decide resolution policy within their remit. Access owners approve audience projections and disclosure tests. Release owners verify the manifest, regression results, invalidation re-

cords, and publication scope. No single automated check can substitute for these decisions. Automation can show that required fields exist and expected cases produce expected outputs.

7.3 Evaluate the Permitted Outputs

Evaluation needs item boundaries before results are inspected. Otherwise an evaluator can improve a score by deciding after the fact that the missing negative assertion did not belong, that an abstention was incomplete, or that an absent citation does not matter. The disclosure contract above now defines what each audience may receive. Fix the evaluation items before inspecting system outputs; the worked passes let us examine those predeclared rules here. The fixture is small enough to adjudicate by hand. That makes it useful for testing mechanics and disclosure behaviour. It cannot estimate accuracy on unseen sources.

Pre-register two separate item types. An evidence-retrieval item specifies the question, source revision, scope, date, assertion pattern, required assertion IDs, required locators, and any required classification. A complete-answer item specifies the question, source revision, scope, date, conservative policy, required answer category, required explanation, required evidence or authorised disclosure, and whether a justified abstention fulfils the task. Score a system against the item type it was asked to solve. A system can retrieve all evidence yet phrase the wrong conclusion. It can also emit the right answer label while hiding a decisive assertion.

For original Q4, the evidence-retrieval item is one unit with two required source-specific assertions, `c1` and `c5`. They count separately only inside that fixed unit. The expected locators are `R1:P1-P2` and `M1:paragraph-3`. The answer item has two determinations. First, whether the records agree. Second, whether version 1 applies under the stated policy. The adjudicated answer is “they disagree” for the first determination and a justified withholding for the second. A reply of “conflicting evidence” with only `c1` is not complete because it has no evidence for conflict.

Work Pass A as a fixed retrieval item before judging the complete answer.

> **Note:** Pass A on original Q4.
>
> **Inputs:** the reference requires `c1` and `c5`. The run returns only `c1`,

so there is one relevant return, one total return, and two required assertions.

Calculation: precision = relevant returns / all returns = 1/1 = 1. Recall = relevant returns / required assertions = 1/2 = 0.5.

Interpretation: every returned assertion is relevant, but half the required evidence is missing. The complete answer fails because that omission changes the policy output from abstention to `supported`.

Pass B retrieves both required assertions and no irrelevant assertion. Precision and recall are each 2/2 = 1. Its complete answer also passes: it names the disagreement, withholds the policy conclusion, and cites both required locators. Retrieval success and answer success remain separate judgements even when this run satisfies both.

Score Pass C against its own R2 and M1 reference. On 15 March, the retrieval item requires `c1@G-C` at `R2:P1-P2` and `c5` at `M1:paragraph-3`, with their classifications. The applicable-evidence sub-item requires one assertion, `c5`, so precision and recall are both 1 divided by 1. The complete-answer item requires two determinations. For the comparison, there is no applicable disagreement and no evidence of agreement because revised `c1` is outside. For applicability, the minutes alone yield `refuted`. On 16 March, the complete-answer item requires the same two citations, a comparison that reports neither source has an applicable assertion, and an abstention on temporal scope because `c5` is indeterminate and `c1@G-C` is outside.

The following pre-registered answer set uses the original source revisions. It has five named items and fixed completeness rules and supplies the inputs for the lab arithmetic.

Item	Required determinations and evidence	Adjudicated result	Complete under the policy?
Q1	Register version for Atlas on 1 March, `c1`, `R1:P1-P2`	Version 1 on the original source	Yes

Item	Required determinations and evidence	Adjudicated result	Complete under the policy?
Q2	Recorded owner on 1 January, continuation to 1 March, `c2`, `R1:P1-P2`	Morgan Lee on 1 January, continuation indeterminate on 1 March	No if completeness demands an unqualified later owner, yes if justified limitation fulfils the item
Q3	Both minutes assertions on 15 March, `c4`, `c5`, `M1:paragraph-3`	Version 2 affirmed and version 1 denied	Yes
Q4	Agreement and applicability on 15 March, `c1`, `c5`, both locators	Disagreement, withhold applicability conclusion	No under a no-abstention coverage rule, yes under policy agreement
Q5	Identity answer for the authorised reviewer or bounded ordinary-user answer	`m-alex` remains unresolved for review, ordinary view returns fixed unavailable status	No under an identity-completion rule, yes under authorised policy agreement

Under a no-abstention coverage rule, only Q1 and Q3 receive complete answers. Coverage is 2 divided by 5, which is 0.4. Under policy agreement, all five items receive their adjudicated outcomes, including the justified limitation in Q2, the withholding in Q4, and the audience-specific bounded response in Q5. Policy agreement is 5 divided by 5, which is 1. Reporting only the latter would imply a broader success than the exercise supports. Report both measures with their definitions, numerators, denominators, source revision, and item boundaries.

Give adjudicators the source version, question, date, scope, policy, required evidence, and rationale prompt before showing system output. Resolve differ-

ing interpretations against retained text and record the decision. Useful pitfalls include no in-scope evidence, malformed rows, unknown ends, required negatives, exclusive boundaries, and repeated upstream sources. Include Q1 at 29 February and 1 March, Q4 at 15 and 16 March, C, and the downstream path. Keep related source families out of both development and evaluation.

When a denominator is zero, report it and leave the ratio undefined. Deduplicate repeated `c1` returns before counting. Retire or version an item when its source revision changes.

The lab demonstrates expected behaviour under specified inputs. It makes no empirical accuracy, recall, safety, or privacy claim. A deployment evaluation needs a larger, independently assembled reference set, source-family splits, adjudicator training, error analysis, confidence intervals where sampling warrants them, and a documented treatment of abstentions and access-controlled outputs.

7.4 Check the Release Regression

Evaluation defines which outputs count as successful. Release regression now checks whether the revised evidence reaches every affected product. Compare `G-B`, with repaired extraction from R1, against `G-C`, with the corrected R2 register, under the same question and audience contracts.

The following regression table compares answer values and provenance across the releases. It includes the boundary dates, the owner and version 2 carry-forwards, the post-minutes date, and the downstream path.

Regression case	`G-B` expectation	`G-C` expectation	Required release check
Q1 on 29 February	Version 1 from applicable `c1@G-B`	Version 1 from applicable `c1@G-C`	Cite sentence 1 in the selected register revision
Q1 on 1 March	Version 1 from applicable `c1@G-B`	No applicable register version. `c1@G-C` is outside and `c6@G-C` is outside before its start	Cite `R2:P1-P2` sentences 1 and 3 for the bounded gap

Regression case	G-B expectation	G-C expectation	Required release check
Q1 on 1 April	Version 2 from c6@G-B at R1:P1-P2, sentence 3	Version 2 from c6@G-C at R2:P1-P2, sentence 3	Check the citation even though the version returned is the same
Q2 owner and continuation	Morgan Lee on 1 January and continuation indeterminate on 1 March, from c2@G-B at R1:P1-P2, sentence 2	Same owner and continuation result from c2@G-C at R2:P1-P2, sentence 2	Check the R2 ownership span and the retained D1 identity evidence
Q4 on 15 March	Disagreement between applicable c1@G-B and c5. Abstain on conflicting evidence	No applicable disagreement and no evidence of agreement. c1@G-C is outside. Minutes alone yield refuted	Cite the selected register interval and the M1 denial
Q4 on 16 March	Register alone has an applicable assertion. c5 is indeterminate. Abstain on temporal scope	Neither source has an applicable assertion. c1@G-C is outside and c5 indeterminate. Abstain on temporal scope	Keep the changed classifications and both source links even where the policy result matches
Version 1 → Atlas → Northbridge path on 1 January	Supported by applicable c1@G-B and c3	Supported by applicable c1@G-C and c3	Rebuild path provenance with the new assertion derivation

Regression case	`G-B` expectation	`G-C` expectation	Required release check
Version 1 → Atlas → Northbridge path on 1 March	Indeterminate because `c3` is indeterminate while `c1@G-B` is applicable	Outside because `c1@G-C` is outside. `c3` is indeterminate.	Do not reuse the old path status or a topology-only projection
Undated positive projection edge version 1 → Atlas	Present	Present	Label the projection as undated and do not use it to answer dated questions

A release can preserve a value while changing its evidence, or preserve an abstention while changing its reason. Q2 and the 1 April Q1 result test the first distinction; Q4 on 16 March tests the second. The undated projection is a control: its unchanged edge is expected and cannot substitute for the dated checks. This is why acceptance compares the intermediate classifications and citations alongside the displayed result.

Complete the release check with the paired disclosure test: compare the approved Q4 citations, fixed ordinary Q5 answer, and visible counts with restricted records present and absent. Check the authorised Q5 review separately, and confirm that an ordinary request cannot obtain the reviewer's cached answer. Every revised answer must identify its release and resolve to the evidence permitted for that audience.

The table and paired tests supply acceptance expectations for this synthetic fixture. A broader graph, live access-control system, or future source feed needs its own coverage, disclosure, and accuracy evidence. With the Northbridge cycle complete, the transfer cases show which modelling decisions carry across domains and which must be defined anew.

8 Transfer the Method to Literature and Dependencies

The same evidence-led method can answer questions about research and software, provided each graph relation keeps its domain meaning. In both cases, a path finds candidate evidence, and explicit conditions decide whether that evidence applies to the question. The cases below are entirely synthetic. Their sources, identifiers, numbers, version rules, and corrections are supplied here so that you can reproduce the results without consulting an external catalogue, registry, or advisory. Neither case establishes a clinical recommendation or a security finding about a real system.

8.1 Literature Evidence for a Research Question

A reviewer asks which studies in a specified review's citation neighbourhood report a reduction in seven-day hospital readmission among adults aged 65 or over after telephone follow-up, compared with usual care. The word "report" sets a modest task. It asks what an identified study result says under the stated conditions. It does not ask whether the intervention should be adopted, whether the result is causal, or whether all relevant studies in the world have been found. Define reduction for this exercise as a negative point estimate of the risk difference, calculated as the telephone group's readmission percentage minus the usual-care group's percentage. An interval around that estimate remains part of the answer because the sign of a point estimate alone does not describe its precision.

8.1.1 Define the Sources and Model

The source boundary is a fictional review and three fictional research works. Each work has a stable identity, and each version is a separately identified source. A work is the intellectual publication across its versions. A version is the particular text from which a claim was extracted. A study is the investigation described by that text, and a cohort is the group of participants whose data are analysed. Two reports can describe the same study, and two distinct studies can draw on the same cohort. Neither relationship is safe to infer solely from similar titles or matching numbers.

The following source catalogue defines the complete initial evidence set. The

quotation in each row is an excerpt of that synthetic source, and the locator identifies the passage available for checking.

Source version	Work and locator	Complete relevant excerpt
R@1	Review R, bibliography entries 8–10	“Entry 8: report A. Entry 9: report B. Entry 10: report C.”
A@1	Report A, methods paragraph 1 and results table 2	“Study A enrolled cohort X, adults aged 65 or over. Telephone follow-up was compared with usual care. Seven-day readmission risk difference was −4 percentage points, with a 95% interval from −7 to −1.”
B@1	Report B, methods paragraph 1 and results table 3	“Study B enrolled cohort Y, adults aged 65 or over. Telephone follow-up was compared with usual care. Thirty-day readmission risk difference was +1 percentage point, with a 95% interval from −2 to +4.”

Source version	Work and locator	Complete relevant excerpt
C@1	Report C, methods paragraph 1 and results table 1	“Study C is a secondary analysis of the same cohort X enrolled in Study A. It compares telephone follow-up with usual care in adults aged 65 or over. Seven-day readmission risk difference was −4 percentage points, with a 95% interval from −7 to −1.”

These excerpts are the entire evidence universe for this exercise at the initial revision. The review supplies three bibliography links. It does not itself establish any outcome, the age of a population, or independence between results. The reports supply those details. Report C explicitly identifies a shared cohort and a secondary analysis, so its separate study and work identities do not justify treating its participants as an independent replication of Study A. The repeated numbers reinforce the need to inspect the relationship, but the explicit source statement is what establishes it here.

Assign identifiers with types before extracting relations. R, A, B, and C are works, R@1 through C@1 are versions, SA, SB, and SC are studies, and X and Y are cohorts. O7 and O30 identify seven-day and thirty-day readmission outcomes. T identifies telephone follow-up and U usual care. The symbols are local to this example. The same visible label in a different collection would need an identity decision before its records were joined.

The next table records the relations extracted from the excerpts. A result assertion is a distinct record because its population, comparator, measurement window, estimate, and interval belong together. Its source locator allows the extraction to be checked against the exact version.

Assertion	Typed subject and relation	Typed object or qualified value	Evidence
L1	Work R cites work	A	R@1, entry 8
L2	Work R cites work	B	R@1, entry 9
L3	Work R cites work	C	R@1, entry 10
L4	Version A@1 reports study	SA	A@1, methods paragraph 1
L5	Version B@1 reports study	SB	B@1, methods paragraph 1
L6	Version C@1 reports study	SC	C@1, methods paragraph 1
L7	Study SA analyses cohort	X	A@1, methods paragraph 1
L8	Study SB analyses cohort	Y	B@1, methods paragraph 1
L9	Study SC analyses cohort	X, as a secondary analysis of SA	C@1, methods paragraph 1
L10	Study SA has result	O7, age ≥65, T versus U, −4 points, interval −7 to −1	A@1, results table 2
L11	Study SB has result	O30, age ≥65, T versus U, +1 point, interval −2 to +4	B@1, results table 3
L12	Study SC has result	O7, age ≥65, T versus U, −4 points, interval −7 to −1	C@1, results table 1

Each `cites` edge means that the review lists a work. It does not mean that the review endorses a conclusion or that the cited work supplies a qualifying result. The `reports` edges connect a particular source version to a study, and the result assertions preserve the conditions under which the estimates were reported. The relation from a work to its version is implicit in the catalogue above and is used explicitly in the join below. A real system would also record extraction and

review status, including rejected or uncertain matches. This exercise treats the shown extractions as accepted and makes no assertion about unlisted material.

8.1.2 Work the Query

To reproduce the query, start with `R` and bind a cited work as `w`. Join `w` to its listed version `v`, then join `v` to a reported study `s`. Join `s` to a result `r` and cohort `c` attested by the selected version `v` in the snapshot. This version binding prevents an earlier result about the same study from entering the revised answer. Keep a row when `r` specifies age 65 or over, intervention `T`, comparator `U`, outcome `O7`, and a risk difference below zero. The join uses work identity to select versions, study identity to select results, and cohort identity to describe overlap. It does not use a citation edge as a substitute for a result assertion.

The intermediate bindings below show every candidate before the outcome and sign filters are applied. The evidence column gives the path and result record needed to inspect each candidate.

w	v	s	c	r and qualification	Evidence path
A	A@1	SA	X	L10, O7, −4, interval −7 to −1	L1, L4, L7, L10
B	B@1	SB	Y	L11, O30, +1, interval −2 to +4	L2, L5, L8, L11
C	C@1	SC	X	L12, O7, −4, interval −7 to −1	L3, L6, L9, L12

All three candidates satisfy the age, intervention, and comparator conditions. `SB` fails both the requested seven-day window and the negative-point-estimate condition. Its thirty-day interval includes zero, so its positive estimate alone is insufficient to characterise the uncertainty of that separate result. `SA` and `SC` pass the stated filter. Grouping by study produces two named studies, while grouping by cohort produces one contributing cohort, `X`. Those are different counts answering different questions. A count of citation paths would also be

two, but it would not establish two independent participant groups.

The evidenced answer at this revision is that SA and SC each report a seven-day risk difference of −4 percentage points, with a 95% interval from −7 to −1, in the specified comparison. A@1 results table 2 and C@1 results table 1 support the respective figures. C@1 methods paragraph 1 identifies SC as a secondary analysis of the same cohort as SA. The answer should therefore name both reported studies and disclose that their evidence overlaps. It should not add their participant counts or present the two records as independent replications. The excerpts contain no sample sizes, allocation procedure, missing-data analysis, or assessment of bias, so they cannot establish how reliable or generalisable either estimate is.

8.1.3 Correct the Selected Source Version

Now suppose an authorised correction produces C@2. The replacement at methods paragraph 1 and results table 1 states “Study C is a secondary analysis of cohort X enrolled in Study A. It compares telephone follow-up with usual care in adults aged 65 or over. The readmission risk difference at thirty days was −4 percentage points, with a 95% interval from −7 to −1. The seven-day label in version 1 was an error.” The correction changes the measurement window of L12 from seven to thirty days. It does not change SC, cohort X, the intervention, comparator, estimate, or interval. C@1 remains an identifiable historical source, and C@2 is the accepted version for the revised evidence snapshot.

The following revision ledger makes the changed evidence and its scope explicit. S1 is the initial snapshot and S2 selects the corrected version of work C while retaining the other three versions.

Snapshot	Selected versions	Result assertion for SC	Status
S1	R@1, A@1, B@1, C@1	L12@1, O7, −4, interval −7 to −1	Historical
S2	R@1, A@1, B@1, C@2	L12@2, O30, −4, interval −7 to −1	Accepted after correction

This ledger prevents the corrected result from silently overwriting the text that supported the earlier answer. It also avoids treating both versions as simultan-

eous independent observations. The revised `reports` and cohort links, `L6@2` and `L9@2`, are supported by `C@2` methods paragraph 1, and the citation from `R@1` still points to work `C`. The review's citation does not need to change when a later version corrects a reported result. A system that stores the version on every assertion must create version-specific supporting links even where their semantic content is unchanged.

Rerun the same query against `S2` with the same bindings and filters. `SA` remains a match through `A@1` and `L10`. `SB` still has a thirty-day result. `SC` now has a negative estimate for thirty days and fails the seven-day filter. The revised answer names only `SA`, with its original estimate, interval, and source locator. One qualifying study and one qualifying cohort remain. A query for negative thirty-day estimates would now admit `SC`, while `SB` still fails the negative-point-estimate condition. Dropping the outcome-window filter would include both `SA` and `SC`, but would mix seven-day and thirty-day measurements and would need to state that change in scope.

Consider how the answer would change if the task asked for papers rather than studies. The selected works in `S1` would be `A` and `C`, each with one qualifying selected version. That gives two works, but it says nothing by itself about how many investigations or cohorts generated their results. In `S2`, only `A` has a selected version with the requested result. The work identity of `C` remains stable through the correction, while the claim extracted from its selected version changes. A catalogue that replaces a work identifier whenever a correction appears would obscure the connection between the two texts and make it harder to explain why an earlier answer changed.

The converse mistake is to attach a result directly to a work without keeping the version and study. Then a reader might see the corrected `C@2` text alongside the obsolete seven-day assertion and have no way to establish which text supported it. Version selection must precede result evaluation. The snapshot is a coherent choice of source versions and accepted assertions, not simply a timestamp on a graph whose contents can change underneath it. A system may implement that choice in several ways, but a reproducible run must identify the same inputs and interpret them under the same rules.

The interval should travel with the estimate even though the example's inclusion rule tests only the point estimate. An interval from -7 to -1 and an interval from -7 to $+1$ would both accompany a negative point estimate, but they would con-

vey different uncertainty under their respective statistical constructions. This example uses the intervals exactly as reported and does not infer a trial design or calculate a new interval. The criterion could be changed to require an interval entirely below zero, provided that change is stated as a new question and applied consistently. Such a filter would be an analytic choice, not a fact conveyed by the citation edge.

This distinction also protects the interpretation of the correction. `C@2` changes the outcome window, so `SC` leaves the seven-day result set while remaining a reported study in the neighbourhood. It does not retract the whole work or establish that the thirty-day estimate is clinically meaningful. The explicit correction text supplies the reason for replacing one qualification. If a new version merely omitted the old table, the absence alone would need interpretation under a declared version and source policy. Preserving the previous passage would still permit a reviewer to inspect the discrepancy and record uncertainty rather than silently choosing a convenient value.

The correction requires more than changing a displayed label. Re-evaluate saved answers, exports, study and cohort counts, and any materialised result or search index dependent on `L12@1`. Check that `S1` still reproduces the historical answer with `C@1`, and that `S2` cannot combine `C@2` with the old seven-day assertion. Check the thirty-day query as well as the original seven-day query because the correction moves a result into a different eligible set. A citation-only query should continue to return all three works. This separates a change in evidence applicability from a change in the review's bibliography.

8.1.4 Audit the Evidence and Interpretation

A reviewer can audit the joins in the opposite direction as well. Start with the answer row for `SA`, locate `L10`, and verify its numerical value, interval, age condition, comparator, and seven-day window in `A@1` results table 2. Then verify the study identity in the methods paragraph and the entry for work `A` in `R@1`. Each step answers a different question. The result passage supports the estimate, the methods passage attaches it to the study and population, and the bibliography establishes why the work belongs to the selected neighbourhood. If the bibliography entry were absent, the estimate could still be present in the report but would fall outside this particular query. If the result passage were absent, the citation path alone could not supply the estimate.

The choice of unit also affects a synthesis beyond this lookup. A study can publish multiple analyses with different windows or populations, and a cohort can appear in multiple studies. Grouping everything by work would overcount publications when versions are revised. Grouping everything by study can still double-count participants when analyses overlap, as `SA` and `SC` show. Grouping by cohort prevents a claim of independent cohorts in this small example, but it does not automatically produce a valid pooled effect. A synthesis would need to determine whether analyses use the same participants, outcome definitions, and follow-up, and then choose a method appropriate to the dependence. The source excerpts do not provide enough information for that calculation.

A result can also fail a filter for a reason that is unknown rather than explicitly false. Here all necessary fields appear in the supplied excerpts, so each exclusion has a stated reason. If a report omitted the age range, its candidate row would have an unknown age qualification. Treating that omission as evidence that the population is younger would be an unsupported inference. A reproducible review would retain the candidate and the missing field, distinguish it from an explicit mismatch, and seek the relevant source passage where the review protocol permits. An answer restricted to confirmed matches would report that distinction instead of concealing it in a zero count.

The result remains bounded by the stated review neighbourhood and the supplied excerpts. An empty result would mean that this procedure found no qualifying result among the selected versions, not that no such study exists. A broader review would need a defined search strategy, screening decisions, independent identity resolution, and a way to represent conflicting or uncertain extractions. In particular, overlapping cohorts can be partial, and this simple shared-cohort label would need participant-level or source-supported overlap information before a reviewer could combine estimates. The graph makes the assumptions and dependencies inspectable, but it cannot supply evidence absent from the sources.

8.2 Installed Dependencies and Advisory Applicability

A maintainer asks which production deployments in a stated inventory snapshot include a package release covered by a particular advisory. The answer needs three distinct facts. A deployment identifies the build in use, a resolved inventory identifies the exact package instances selected for that build, and an advis-

ory defines which releases it covers. A declared version constraint describes a set of permitted releases. It does not establish which member of that set was selected or deployed. A path from a production build through its resolved dependency edges supplies that selection, subject to the completeness and timing of the inventory.

8.2.1 Define the Inventory and Model

Everything in this example is fictional, including the package names, registry, releases, inventory format, and advisory. Use the following explicitly synthetic version rule throughout. Versions have three non-negative integer components ordered numerically from left to right. A constraint written `>=a <b` includes versions at or above `a` and strictly below `b`. There are no prereleases, platform variants, aliases, or other version rules in this exercise. An exact release is identified by the tuple of registry, package name, and all three version components. The labels and arithmetic do not describe any real package ecosystem.

The source boundary comprises a deployment record, a manifest, two resolved inventories, and an advisory. The question is evaluated at the named snapshot, and its answer remains attached to those selected source versions. This makes the conclusion reproducible even after a later record changes the evidence available to the maintainer. Each row below gives all source text needed for the initial question, with stable source versions and locators.

Source version	Locator	Complete relevant excerpt
`DEP@1`	deployment rows 1–2	“Production deployment P uses Portal build 41. Development workspace D uses Portal build 42.”
`MAN@1`	dependency row 1	“Portal permits registry `F`, Parser `>=2.4.0 <3.0.0` for runtime.”

Source version	Locator	Complete relevant excerpt
`I41@1`	inventory rows 1–2	“Build 41 runtime selects `F/Parser/2.4.0` directly. That selected Parser instance selects `F/Codec/1.8.0` at runtime.”
`I42@1`	inventory rows 1–2	“Build 42 runtime selects `F/Parser/2.4.0` directly. That selected Parser instance selects `F/Codec/1.9.0` at runtime.”
`ADV@1`	affected and fixed sections	“Advisory V covers registry `F`, Codec releases `>=1.8.0 <1.9.0`. Release `1.9.0` is fixed for V.”

The manifest permits Parser 2.4.0 under the synthetic ordering, but it permits other Parser releases in the range too. It does not name Codec or determine a Codec selection. The inventories supply those selections for their respective builds, and `DEP@1` supplies the production or development context. `ADV@1` describes an affected release range. It does not state that an affected release is installed anywhere, that an affected execution path is reachable, or that a particular deployment has been compromised.

Give the production deployment the identifier `P`, the development workspace `D`, and the builds `B41` and `B42`. A package release identifies a registry, name, and version. A selected instance additionally identifies its build and resolution context. Thus `P41` and `P42` both refer to release `F/Parser/2.4.0` but are distinct selected instances. `C41` refers to `F/Codec/1.8.0` in build 41 and `C42` to `F/Codec/1.9.0` in build 42. This distinction allows the same Parser release to have different resolved children in different builds without merging those children into one global dependency list.

The extracted facts below carry the context required for a scoped traversal. `runtime` is an edge scope supplied by the inventories, and the build identifier is part of each selection or dependency fact.

Assertion	Typed relation	Qualification	Evidence
`D1`	Deployment `P` uses build `B41`	production	`DEP@1`, row 1
`D2`	Workspace `D` uses build `B42`	development	`DEP@1`, row 2
`D3`	Build `B41` selects instance `P41`	runtime, direct, release `F/Parser/2.4.0`	`I41@1`, row 1
`D4`	Instance `P41` selects instance `C41`	build `B41`, runtime, release `F/Codec/1.8.0`	`I41@1`, row 2
`D5`	Build `B42` selects instance `P42`	runtime, direct, release `F/Parser/2.4.0`	`I42@1`, row 1
`D6`	Instance `P42` selects instance `C42`	build `B42`, runtime, release `F/Codec/1.9.0`	`I42@1`, row 2
`D7`	Advisory `V` covers releases of package	registry `F`, Codec, `>=1.8.0 <1.9.0`	`ADV@1`, affected section
`D8`	Advisory `V` identifies a fixed release	registry `F`, Codec, `1.9.0`	`ADV@1`, fixed section

The dependency from Parser to Codec is transitive from the build's perspective. The build selects Parser directly, and Parser selects Codec. The path is valid only when all its edges belong to the same build and requested runtime scope. A graph keyed only by the Parser release would merge `D4` and `D6`. Traversing that merged node could falsely attach Codec 1.8.0 to build 42 and Codec 1.9.0 to build 41. Selected-instance identifiers or equivalent edge qualifiers prevent that cross-build join.

8.2.2 Traverse and Test Advisory Applicability

To answer the question, bind a deployment `d` from `DEP@1` whose environment is production. Follow its `uses build` edge to `b`, then traverse direct and transitive `selects` edges within `b` and runtime scope. Bind each reached instance as `i` and its exact release as `p`. Join `p` to advisory `V` only if the registry and package name match and the numeric version falls within the advisory range. The path may have more than one dependency edge in a larger inventory. Here the complete supplied paths have at most two. Keep the full path and the inventory and advisory versions with each result.

The next table displays the candidate paths before the production and advisory filters are combined. The version comparisons use the synthetic rule stated above.

Context and build	Resolved path	Reached Codec release	Advisory comparison
Production `P`, `B41`	`B41` → `P41` → `C41`	`F/Codec/1.8.0`	`1.8.0` ≥ `1.8.0` and `1.8.0` < `1.9.0`, so covered
Development `D`, `B42`	`B42` → `P42` → `C42`	`F/Codec/1.9.0`	`1.9.0` equals the exclusive upper bound, so outside the affected range and explicitly fixed

The initial answer is `P` and build `B41`, with the path supported by `DEP@1` row 1, `I41@1` rows 1–2, and `ADV@1` affected section. The development path is useful as a control for the version boundary, but it is outside the production question in any event. The Parser instance itself does not match `D7` because the package name is Parser rather than Codec. Matching version numbers without checking registry and package identity would also create false results. The manifest shows that the selected Parser release satisfies its declared constraint, but the manifest alone cannot establish the transitive Codec release.

This answer is an inventory-and-advisory finding for a specified snapshot. It establishes that an affected release appears on a recorded runtime dependency

path in a production build. It does not establish that a vulnerable function is invoked, that a particular configuration satisfies additional conditions, or that an exploit occurred. The synthetic advisory has no additional applicability conditions, and the inventory is assumed complete within its declared runtime scope. Real advisories and inventories may have narrower conditions, missing edges, platform distinctions, and collection delays. Those facts would need separate evidence and explicit predicates before a stronger conclusion.

8.2.3 Correct the Inventory and Replay

Now correct the production inventory. Version `I41@2` retains row 1 and replaces row 2 with the complete statement, "That selected Parser instance selects `F/Codec/1.9.0` at runtime. The 1.8.0 selection recorded in inventory version 1 was an error." This revises the evidence about the same build and deployment. The selected Codec instance in the corrected inventory is `C41b`, bound to build `B41` and release `F/Codec/1.9.0`. The revised snapshot has an assertion record for each hop, including the direct selection whose wording was carried forward.

Assertion	Typed relation	Qualification	Evidence
`D3@2`	Build `B41` selects instance `P41`	runtime, direct, release `F/Parser/2.4.0`	`I41@2`, row 1
`D4@2`	Instance `P41` selects instance `C41b`	build `B41`, runtime, release `F/Codec/1.9.0`	`I41@2`, row 2

The snapshot ledger records the inputs for the original and corrected runs. The deployment record, development inventory, and advisory are unchanged.

Snapshot	Evidence versions	Build `B41` runtime Codec path	Advisory `V` result
`D-S1`	`DEP@1`, `MAN@1`, `I41@1`, `I42@1`, `ADV@1`	`B41` → `P41` → `C41`, release `1.8.0`	Covered

Snapshot	Evidence versions	Build B41 runtime Codec path	Advisory V result
D-S2	DEP@1, MAN@1, I41@2, I42@1, ADV@1	B41 → P41 → C41b via D3@2 and D4@2, release 1.9.0	Outside range, explicitly fixed

This ledger gives the correction a precise effect. The production deployment still uses B41, and the path still goes through Parser 2.4.0. Its second edge now reaches Codec 1.9.0, whose version equals the exclusive upper bound and is named as fixed in ADV@1. Repeating the original production query against D-S2 returns no matching deployment in this complete synthetic source boundary. The answer is therefore that the corrected inventory supplies no production path to a release covered by V. That statement remains conditional on the snapshot and advisory. It does not establish that the production environment is free of other advisories or that a later deployment has the same contents.

The change invalidates the old finding's dependency on D4 as accepted evidence. Recompute affected paths, production counts, saved answers, exports, and caches that used I41@1. Confirm that the historical D-S1 run still yields the old path and identifies its superseded inventory, while the accepted D-S2 run does not join the old Codec edge to the corrected snapshot. Recheck B42 as a control because its inventory did not change. Also recheck the boundary comparison for 1.9.0, the registry and package equality tests, and the production filter. An unscoped traversal that still returns Codec 1.8.0 for B41 reveals a stale edge or an improperly constrained join.

8.2.4 Audit Scope, Boundaries, and Missing Evidence

The order of the traversal matters for both accuracy and explanation. Starting with production deployments limits the relevant builds before package paths are expanded. Keeping the build binding at every hop prevents an edge from a different resolution being borrowed merely because the parent release has the same name and version. After reaching a candidate, the advisory join evaluates the exact release tuple. A path can be valid and still fail the advisory test, as the B42 path demonstrates. Conversely, a release can satisfy the advisory range without being reachable from any deployment in the snapshot. Such a release describes potential applicability, not an installed finding.

Scope is equally important. The inventory calls both edges runtime dependencies, and the question selects that scope. If an inventory also contained a test-only edge, the same package release could appear in the build's development context without meeting this query. A different question about build tooling could deliberately select that scope. The reader should preserve the distinction in the answer and in the evidence path. A generic edge with the scope erased would make it impossible to explain why the release was included or excluded, even if the final count happened to be correct.

The range comparison can be checked without relying on a package manager. Compare major components first, then minor components when majors match, then patch components when both earlier components match. For `1.8.0`, the lower bound is equal and the minor component is below the upper bound's 9, so both conditions hold. For `1.9.0`, equality with the upper bound makes the strict less-than condition false. The advisory's explicit fixed statement supports the same classification for this exact release. It does not imply that every version outside the affected range has been assessed or is safe in every respect. The rule for this exercise is deliberately complete only for the listed numeric releases and the stated advisory.

An inventory's completeness is a property of its collection and declared scope, not a property that follows from the shape of the graph. The example stipulates that each supplied inventory lists the runtime selections relevant to its build. This permits the empty corrected result to cover the full synthetic production boundary. In a larger investigation, a missing package edge could mean that the package was absent, that collection failed, or that a component was outside the inventory's scope. Those possibilities should be represented separately when the evidence permits. A path query can only traverse edges present in the selected snapshot.

The same care applies to an advisory. Here `ADV@1` states a complete synthetic affected range for the named package, and the comparison has no extra predicates. Another advisory could depend on platform, configuration, or a particular feature. In that setting a matching release would establish only the version part of the applicability test until the remaining facts had been checked. Unknown configuration would remain unknown. A system that collapses unknown into either affected or unaffected would hide a material qualification, while a system that records the unsatisfied evidence requirement can ask for the relevant

configuration later.

The inventory correction also illustrates why a record should keep both the source's assertion and its acceptance status. At `D-S1`, `I41@1` supports the finding shown for that historical run. At `D-S2`, its superseding version supports a different selected release. Keeping `I41@1` does not mean treating the superseded selection as an alternative current edge. The snapshot selects one accepted account for this purpose. If two independent inventories disagreed without an authorised correction, the graph would need to retain both claims and their provenance until the discrepancy was resolved. The present example does not silently choose between competing sources because its replacement statement explicitly identifies the earlier error.

A reviewer can test the result by starting from the evidence cited in the answer and tracing backwards. The affected-range passage identifies `F`, Codec, and the bounds. The inventory row identifies the exact Codec release and the selected Parser instance in the same build. The direct-selection row connects that Parser instance to the build, and the deployment record connects the build to production. Removing any one of these links weakens the conclusion in a specific way. For example, losing the deployment record leaves a build-level finding without evidence that the build belongs to a production deployment. Losing the advisory passage leaves an installed-release observation without evidence that the release falls in this advisory's range.

A practical audit should also inspect negative results rather than counting only returned rows. Under `D-S2`, trace `P` to `B41`, then `P41`, then `C41b`, and check why the final advisory comparison fails. This establishes that the empty result follows from a present, out-of-range candidate. An empty result caused by a missing inventory would have a different meaning. If `I41@2` were unavailable, the system could not establish the selected transitive release from `MAN@1` and `DEP@1` alone. The appropriate status would be unresolved for that build, with the missing evidence recorded, rather than an assertion that the build has no affected component.

A later advisory correction would require a separate evidence version and rerun even if neither inventory changed. For example, if an explicitly revised advisory expanded the covered range, each reached Codec release would be tested against that new range using the same synthetic comparison rule. The result would depend on the actual replacement text, which this example does not sup-

ply. Likewise, a genuine redeployment would require a new deployment record and a resolved inventory for the new build. It should not be represented as a correction to the old build's historical contents. Recording which input changed distinguishes an amended observation from a change in the world.

8.3 Compare the Modelling Decisions

The two cases share a workflow, but their admissible joins depend on different units and qualifications. Use this comparison when transferring the method to a new corpus.

Modelling decision	Literature evidence	Dependency evidence
Entity grain	Keep work, source version, study, and cohort distinct; two studies can share participants.	Keep package release and selected instance distinct; one release can have different children in different builds.
Qualifications	Preserve population, intervention, comparator, outcome window, estimate, and interval together.	Preserve build, environment, runtime scope, exact release tuple, and advisory range.
Admissible joins	Select a work's version, then use study and result assertions attested by that version.	Keep the build and runtime binding at every dependency hop, then test the reached release against the advisory.
Completeness	The supplied review neighbourhood is complete only for this exercise; it cannot establish all relevant research or independent cohorts.	The stipulated runtime inventories and affected range permit the bounded empty result; an unavailable inventory would leave it unresolved.

Modelling decision	Literature evidence	Dependency evidence
Correction dependencies	`C@2` changes the eligible window; rebuild version-specific links, result sets, study/cohort counts, and their citations.	`I41@2` changes a resolved child; rebuild both inventory hops, affected paths, deployment findings, and their citations.

A reproducible answer also states the rule used to resolve each boundary. In the literature example, the seven-day label is an exact match and a negative point estimate satisfies the stated reduction criterion. In the dependency example, the upper version bound is exclusive and the environment must be production. These choices belong to the question and its evaluation record. Changing one may produce a different valid answer, so a reader comparing runs needs the rule as well as the selected sources. The evidence path explains which records contributed to a result, while the failed candidates explain why other records did not. Together they make an apparently simple count open to inspection and correction. They also give the next reviewer a concrete starting point for checking the result.

Both cases turn a broad question into a scoped path and an evidence test. In the literature case, the outcome window and cohort relationship decide what a reported result contributes. In the dependency case, the build context and advisory range decide whether a selected release qualifies. A path alone supplies neither a clinical conclusion nor proof of runtime exploitation. In each case, retaining the source version, qualifiers, and query dependencies makes a correction traceable and lets the same question be answered again against a named snapshot.

9 Analyse a Positive Projection

The evidence ledger is designed for questions about what a source said and when its assertion can be used. Sometimes the question is structural. For example, can you reach Northbridge by following outgoing arrows from version 1, and how many separate groups of nodes appear in the network? A projection selects and transforms the ledger for such a purpose. Its rules specify which vertices

and relationships enter the graph, what they retain, and what can be recovered from them.

9.1 Construct the View

Use all eight entity IDs from Northbridge as vertices, including the two directory people called Alex Kim. Select positive assertions regardless of date or source. Group the selected assertions by the ordered triple of subject, predicate, and object, then emit one edge per distinct triple. Assign each edge a cost of one for counting steps. A group can have more than one member. Keep the assertion IDs alongside the calculation below so you can check the transformation. The actual five-edge projection retains the relation type and cost and discards those IDs.

The grouping produces these edges from the original seven assertion rows.

Directed edge	Relation	Positive inputs
Version 1 → Atlas	`applies_to`	`c1`
Version 1 → Morgan Lee	`owned_by`	`c2`
Atlas → Northbridge	`belongs_to`	`c3`
Version 2 → Atlas	`applies_to`	`c4`, `c6`
Orion → Northbridge	`belongs_to`	`c7`

There are seven assertion rows, six positive assertion rows, five distinct positive triples, and six vertices that occur at one or more edge endpoints. The two remaining vertices come from the declared vertex set. Without it, an edge list would give a graph with six vertices and no visible isolates. An isolate is a vertex with no incident edges in the chosen view. That says something about the edge selection, and supplies no evidence about the person's activity. Including the Alex vertices is part of the fictional structural exercise. An audience view would still have to authorise the vertices themselves.

The rule also explains why the two version 2 assertions become one edge. Their sources and valid times differ, but the triple key is the same. Counting edges measures distinct connections under this choice. Counting six positive source assertions would measure the size of a different object. Dropping the relationship type from the key could collapse distinct relations between the same participants. Dropping direction could collapse two relations facing opposite ways.

A projection should declare those choices explicitly so that someone comparing counts can recover the unit.

9.2 Work the Path and Component Calculations

For the shortest directed path, start at version 1 with cost zero. At cost one, its outgoing edges reach Atlas and Morgan Lee. Neither is Northbridge. Expand both vertices one step. Morgan Lee has no outgoing edge and Atlas reaches Northbridge. The first time Northbridge appears is at cost two, with path version 1 → Atlas → Northbridge. With unit costs, exploring by increasing number of edges guarantees that the first hit is a shortest path. There is no direct version 1 → Northbridge edge in the projection. If you start instead at Northbridge and follow only outgoing edges, you cannot reach version 1 because Northbridge has no outgoing edge.

Unit weights are a modelling choice. Suppose the task assigned a cost of four to the applicability hop and one to the affiliation hop. The same path would then cost five. On a larger graph, taking the first hit by number of edges could overlook a longer path with a lower cost. Give weights a defined meaning and use a procedure suited to that meaning. The costs in this example count hops and have no evidential or organisational interpretation. A confidence rating would need a justified rule for combination, especially when two edges depend on one source.

For **weakly connected components**, ignore direction when testing whether nodes are connected. Start from Northbridge. You can reach Atlas and Orion by following their incident edges. From Atlas you can reach both policy versions, and from version 1 you reach Morgan Lee. This is a component of six. Each Alex entity is isolated, so there are three weak components. The direction-sensitive notion of **strong connectivity** instead requires mutual reachability by directed paths. This particular projection has no directed cycle and every node is its own strong component. The component label should state which definition produced it.

The node table can summarise the same edge list. It keeps one row per vertex and counts edges pointing in and edges pointing out.

Vertex	In-degree	Out-degree	Total degree	Weak component
Version 1	0	2	2	Main
Version 2	0	1	1	Main
Atlas	2	1	3	Main
Morgan Lee	1	0	1	Main
Orion	0	1	1	Main
Northbridge	2	0	2	Main
Alex Kim, NB-201	0	0	0	Singleton
Alex Kim, NB-202	0	0	0	Singleton

The in-degrees sum to five and the out-degrees also sum to five because each directed edge contributes once to each count. Atlas has the largest total degree, three, under this projection. Keeping two separate edges for `c4` and `c6` would raise Atlas's degree without bringing in a new neighbour. Connecting Orion to Atlas through their shared affiliation could also change degree if a derived relation were added. Degree reports the selected structure and should be interpreted alongside the edge selection.

9.3 Return From Topology to Evidence

Both the original and corrected registers yield the same undated positive triple for version 1 and Atlas. The path and degree table therefore stay the same after the correction if the projection is recomputed under the same undated rule. Yet the revised `c1` is outside on 15 March, and the minutes' denial has no positive applicable counterpart under the conservative policy. A topology result that stayed the same would be expected. An evidence query that stayed the same would require diagnosis.

If an analyst needs a 15 March register-only positive view, select the register and classify time before projecting. The original register contributes the version 1 edge on that day. The corrected register does not. For a cross-source decision view you would have to group matching assertions and apply the conflict policy before deciding what edge to display. The original R1 and M1 evidence would yield a conflict for version 1 on 15 March. Adding only supported positive edges would omit that version, so a missing edge in that view could mean conflict, indeterminacy, exclusion by scope, or no evidence. Preserve the omitted-edge

classifications if readers will interpret absences. For a dated path, each hop needs its own source and day support even if a projection was labelled with a date.

The R companion illustrates the topology operations on the five declared edges and eight vertices. In `igraph`, the edge table supplies the endpoint IDs and the separate vertex table preserves the isolates. The analysis follows outgoing edges for the path and ignores direction for weak components. A `tidygraph` view selects the node table when adding degree and component membership so the resulting fields describe vertices. The paper calculation here supplies the expected results and their interpretation. igraph graph construction and tidygraph table selection.

10 Practise the Reasoning

Use the original Northbridge evidence unless an exercise specifies a correction. Write the binding or source selection and at least one intermediate step for each answer. Then compare your explanation with the worked answer. An appropriate abstention should identify the condition that prevents a stronger result.

10.1 Questions

1. **Owner and date.** Evaluate Q2 at the 1 January register snapshot and test continuation on 1 March. Identify the endpoint you fix, the endpoint you return, and the evidence needed for each step. What would go wrong if you asked for a version in the object role?
2. **Boundary and disagreement.** Classify `c1` and `c5` for 15 and 16 March. Now suppose a register correction gives 15 March itself as the exclusive end of `c1`. Repeat both days, keeping the minutes and the answer policy fixed.
3. **A qualified path.** Work the version 1 → Atlas → Northbridge path on 1 January and 1 March under the original register. Repeat after the lab correction to an exclusive end of 1 March. Explain whether any answer supplies version 1 applicability for Orion.
4. **Missing evidence or a bad answer.** A Q4 response on 15 March says “The records disagree” and cites only the register. A diagnostic trace shows that `c1` and `c5` both reached the decision with the correct classifications. Where

is the defect? How would your diagnosis change if only `c1` reached the decision even though both assertions were admitted?

5. **An identity leak.** An ordinary view masks both Alex labels and IDs. It still allows a user to count paths from a meeting to an attendance mention and then to identity candidates. Compare the outputs if the restricted matching decision is present or absent. What would you test after changing the view?
6. **A source-version join.** In the literature example, run the seven-day negative-estimate query with `C@2` selected. Then run it without the measurement-window condition. If the graph returns the old seven-day result in the corrected snapshot, identify a join that could explain the failure. Compare counts by study and cohort in the second run.
7. **Context at every dependency hop.** In the original dependency snapshot, merge the two Parser instances on their release identifier before looking for Codec. Which false path can you now traverse in each build? In the corrected snapshot, suppose the production inventory is unavailable. Explain why the empty result has a different meaning from the worked corrected result.
8. **Evidence score and projection.** For original Q4 on 15 March, a retrieval system returns `c1`, `c5`, and `c4` as its three applicable evidence items for version 1. Compute precision and recall on the stated item. What result could a positive-only topology produce on the same inputs, and can it receive a correct Q4 answer score?

10.2 Worked Answers

1. Bind version 1 as the subject of `owned_by` and the register as source. `c2` binds the object to Morgan Lee. The assertion has an unknown end and supports 1 January, so report Morgan Lee as the register's recorded owner at the snapshot with `R1:P1-P2`. The source mention's directory ID establishes the identity link in this case. It supplies no extra days for the ownership relation. The date 1 March exceeds the supported horizon, so continued ownership is indeterminate. The object of `owned_by` is a person. Binding a policy version there would find no valid matching row and tells you nothing about whether a recorded owner exists.
2. The original `c1` is applicable both days. `c5` is applicable on 15 March and indeterminate on 16 March. The sources disagree on the first day. Only the register has an applicable assertion on the second. The policy abstains on

both days, first for conflict and then for temporal scope. With 15 March as the exclusive end, `c1` is outside both days. On 15 March the sources have no applicable disagreement and there is no evidence of agreement. The minutes denial alone yields `refuted`. On 16 March neither source has an applicable assertion, and the policy abstains because `c5` is indeterminate.

3. Join the object of `c1` to the subject of `c3` on `project:atlas`. With the original register both hops are applicable on 1 January. On 1 March the second is indeterminate. After the correction the first hop still covers 1 January. On 1 March the first hop is outside and the second remains indeterminate. The whole path is outside because one required premise fails the date. The `c7` affiliation for Orion has a different subject. Nothing in the path rule adds applicability to another project sharing an organisation.
4. The decision received the correct evidence. Inspect how the answer attaches evidence locators. For the ordinary answer, require `M1:paragraph-3:sentence-2` alongside `R1:P1-P2:sentence-1`. If an admitted `c5` never reached the decision, inspect the source and polarity selection and its intermediate outputs. A sentence can be retained, correctly interpreted, and successfully admitted, then lost during retrieval. Repair that stage and test both selected rows before evaluating the answer presentation.
5. The unrestricted calculation returns two paths in the first version and zero in the second. Masking labels leaves the existence and multiplicity observable. In the ordinary view the computation should never use the restricted mention or candidate edges. Compare the outputs of existence tests, counts, search, evidence lookups, exports, and cache retrieval with and without the restricted decision. The Q5 response in both states is “Attendance identity is unavailable in this view.” An authorised review should still be able to distinguish the two inputs, with `r2` and its evidence available in the first.
6. Under the corrected literature snapshot, only Study A passes the seven-day filter. Without that filter, Study C passes with its negative thirty-day estimate. Study B still fails the negative-estimate condition. The second run gives two studies and one cohort, X. Joining results on study identity alone can attach the old `L12@1` row even after `C@2` was selected, if the result lookup can see historical rows. Check that the result is supported by the selected version in that snapshot. Changing the measurement-window condition has made the outcomes heterogeneous even though both estimates

have the same sign and magnitude.

7. A merged Parser node can carry both Codec edges. Build 41 can incorrectly reach the 1.9.0 instance from build 42 and build 42 can incorrectly reach the 1.8.0 instance from build 41. If the query is expanded to development, the second error creates a false advisory match. The first can also conceal the production finding if the system takes just one reachable release. Bind build and runtime scope at each hop. In the corrected example, the present inventory positively identifies Codec 1.9.0, outside V's range. An unavailable inventory gives no evidence of the selected transitive version. The manifest's allowed Parser range supplies no substitute for that evidence.
8. The gold items are `c1` and `c5`. The two correct returns out of three give precision 2/3, and both gold items were returned, giving recall 2/2. `c4` concerns version 2 and is outside the fixed proposition even though the minutes assert it on the same date. The positive topology can show version 1 → Atlas and version 2 → Atlas. It discards the negative evidence and its temporal scope, so these edges cannot support the complete answer about agreement. For an answer score, require the correct judgement, the justified abstention on applicability, and the evidence for both opposing assertions.

11 Storage and Implementation Reference

The ledger can be represented in several ways. The distinction to carry across a mapping is between a relationship that the graph asserts directly, an identified record of someone else's qualified assertion, and an analytical edge derived from selected evidence. Use `c1` and `c5` as a test. A reader of the new representation should still be able to tell that they concern the same proposition, differ in polarity and time, and originate from different sources.

11.1 RDF, Evidence Resources, and Constraints

The Resource Description Framework (RDF) models a graph as a set of triples. Each triple has a subject, predicate, and object. In RDF 1.1 a subject is an Internationalised Resource Identifier (IRI) or blank node, the predicate is an IRI, and the object can also be a literal value. An IRI gives a globally scoped name. A blank node can represent a resource without assigning such a name. A literal can carry a datatype or language tag. An assertion that needs to be cited across

releases benefits from a stable, governed IRI. Repeating the same triple in one graph leaves the set unchanged, so duplicates cannot carry separate evidence. RDF 1.1 Concepts and Abstract Syntax.

Map each source-specific assertion to an identified resource. For the two applicability rows, use a local vocabulary with links to the version, project, and source, and properties for polarity, start, end kind, and whichever end or horizon the row needs. The table shows how the assertion resources can carry the comparison.

Assertion resource	Version and project	Source reference	Qualifications
Resource for `c1`	Version 1 and Atlas	R1, `R1:P1-P2`	Positive, bounded, start 1 January, exclusive end 1 April
Resource for `c5`	Version 1 and Atlas	M1, `M1:paragraph-3`	Negative, unknown end, start and horizon 15 March

In this mapping an assertion resource has one relationship to each of its participants, so it can represent a relation with several roles and qualifiers. An implementation may give the qualified claim its own resource and point source assertions to it. Use the same claim resource only where the qualifiers have the intended common meaning. Adding a bare version 1 applicability triple to the graph would assert that relation under the vocabulary of the added triple. The assertion resource can describe the source's affirmation without committing the graph to unqualified applicability. Named graphs can organise source versions or views, while the meaning of graph names and the evidence linking individual claims to spans still need a documented convention. W3C note on n-ary relations.

SPARQL 1.1 matches triple patterns with shared variables. A pattern binding the assertion, version 1, and Atlas can yield both resources, then follow their source and locator links. Treat the end and horizon as optional bindings whose interpretation depends on the required end kind. A missing optional value may be expected, while a missing required source can cause a mandatory pattern to drop

the assertion. Search for malformed records separately. The Shapes Constraint Language (SHACL) can check conditions such as one version, project, and start per applicability assertion. The selected shapes need additional conditions for the combinations of end fields and date order. Conformance to those shapes gives evidence about structure. Verification of the sentence requires inspecting the source. SPARQL 1.1 Query Language and Shapes Constraint Language.

The PROV Ontology (PROV-O) describes production history using entities, activities, and agents. A retained M1 snapshot and the admitted `c5` assertion can be entities. The extraction is an activity that used one entity and generated another. A responsible software process or organisation can be associated as an agent, and a review can be recorded as a further activity. Use separate vocabulary for 15 March as the asserted day and for the minutes' evidential role. Generation of the assertion in an extraction activity establishes neither its valid time nor its authority. PROV-O.

11.2 Property Graphs and Serving Choices

A property graph can store both `c1` and `c5` as parallel directed relationships carrying their assertion IDs, source references, locators, polarities, and time fields. Cypher pattern matching can find relationships joining version 1 to Atlas. Retain both matches before classifying time or aggregating the evidence. Where an assertion itself must connect to multiple supporting sources, reviews, and correction decisions, it can be clearer to give it a node and link it to those records. Test the representation by asking whether the change to `c1` can be traced independently of `c5`. A node label or a relationship type alone provides little assurance that an import preserved those records. Cypher pattern matching.

Storage choice depends on the work the system must support. Tables can hold all of the evidence and handle direct patterns and fixed-depth paths through joins. An RDF representation may be valuable when interoperable identifiers, shared vocabularies, RDF interchange, or SPARQL queries drive the choice. A property-graph engine can be worth testing for a service that repeatedly explores variable-depth neighbourhoods. In-memory projection is sufficient for the eight-node analysis. Each approach can represent an appropriate view. Compare the entire cost of serving and correcting the specified workload, including graph degree, traversal depth, updates, access constraints, rebuilds, and concurrent requests. A small structural example provides no ranking of engine

performance.

Use an exchange test before drawing performance conclusions. Export the original ledger, import it into the candidate representation, and retrieve the same two applicability assertions, their valid-time fields, and their evidence. Repeat Q4 on 15 and 16 March. Then import the corrected register as a new revision and verify that the original release can still be queried where retention allows. Include the unresolved mention and the audience test, since preserving graph structure alone does not preserve access semantics. A comparison of node and edge counts could pass while every source locator and horizon was lost. Where an export intentionally loses information, document the view's question and carry an evidence mapping if users must return to the original.

11.3 Companion Implementation Notes

The local tabular representation in the Python and DuckDB companion shows how these relations can be persisted and queried after reopening a database. Assertions, sources, mentions, and source-derived extraction evidence occupy separate tables. The assertion's source key supports the required evidence join and the mention key supports an optional one. For the example's source-derived rows, the extraction evidence records exact spans and the method. A query parameter supplies the requested date as a value. Closing the connection and reopening read-only exercises persistence of the records used by the queries. The example uses temporary databases and its validation checks operate in the construction program. The persisted schema alone does not enforce the full ledger contract. DuckDB Python database interface.

The companion constructs `c1`, `c4`, and `c5` from synthetic excerpts and combines them with curated rows. Its bounded correction run omits the negative candidate, restores it, and rebuilds after the hypothetical changed register end. The complete Q2 answer, the audience views, and the transfer cases are hand-worked in this primer. The R companion reads a precomputed positive projection. It does not receive the changed source and its rebuilt assertions, so using R for a dated analysis after a correction would require constructing the new, explicitly scoped projection first. The examples demonstrate those bounded operations. Durable release publication, live access enforcement, and service migration require designs and tests beyond the companions.

To implement the worked route, use its answer records as acceptance criteria.

First compare selected assertion IDs, source revisions, and time classifications; then compare policy results and authorised evidence. Reopening a database tests persistence, while repeating those comparisons tests whether the representation retained the distinctions the question needs. When a topology projection is exported, record its selection rules and discarded qualifiers with it. These checks let an implementation be assessed against the standalone reasoning even when its tables or query language differ.

12 Further Reading

- The W3C note on n-ary relations works through how to identify a relationship when several participants or qualifiers belong together.
- The W3C PROV data model develops the distinction between what was produced, the activity that produced it, and who was responsible.
- RDF 1.1 Concepts and SPARQL 1.1 provide the formal model and query behaviour behind the reference mapping. The definitions of datasets and optional graph patterns are especially relevant to interpreting omissions.
- Estimating the false negatives due to blocking in record linkage examines what happens when a true match is excluded during candidate selection.
- NIST SP 800-188 explains disclosure risk and governance for data sharing, including why linkability matters even after removing direct identifiers.